

SCRBenchmark

Plateforme de benchmarking pour le clustering
de données scRNA-seq

Fabien Bidet

Université de Bordeaux – LaBRI
Stage de Master 2

17 juin 2026

Résumé

Ce rapport décrit SCRBenchmark, une plateforme pour l'évaluation et la comparaison d'algorithmes de *clustering* appliqués aux données de séquençage d'ARN sur cellule unique (scRNA-seq). Le logiciel s'appuie sur un registre extensible d'algorithmes (méthodes classiques, *deep learning* et variantes), incluant notamment PCA+K-Means, PCA+Leiden, scDeepCluster, scMAE (base/small/tiny), scNAME, scHNTL, scMUG, scCDCG, scVI-DCT, Enhanced Autoencoder et scProto (avec C2S+Leiden disponible selon dépendances). Les performances sont évaluées avec des métriques standardisées (NMI, ARI, ACC, Silhouette), complétées par des métriques de robustesse (UCA, F1_Macro, BalancedACC, RareACC, KNN Purity, NNO). SCRBenchmark propose un mode *benchmark* avec séparation train/test stratifiée, une correction de *batch effects* via scVI (composantes DCT/CorrMSE, en modes supervisé et non-supervisé), et des mécanismes de validation automatique (« guardrails »). Parmi les configurations testées, le pipeline scVI-DCTCorr + Enhanced Autoencoder + Leiden obtient les scores les plus élevés sur le dataset Baron Human Pancreas, combinant modélisation probabiliste, représentation profonde et détection de communautés par graphe.

Table des matières

1 Introduction

SCRBenchmark est une plateforme unifiée pour l'évaluation et la comparaison d'algorithmes de clustering appliqués aux données de séquençage d'ARN sur cellule unique (scRNA-seq). Ce rapport présente en détail l'architecture, les métriques d'évaluation, les algorithmes implémentés et les méthodes de benchmarking.

1.1 Objectifs du Logiciel

- **Comparaison objective** : Évaluer plusieurs algorithmes sur les mêmes données avec des métriques standardisées
- **Généralisation** : Mesurer la capacité des modèles à gérer des données non vues (mode benchmark train/test)
- **Ajustements légers** : Utiliser la validation pour l'early stopping / scheduler de LR et des variations légères de paramètres sensibles
- **Reproductibilité** : Permettre la réplication exacte des expériences via CLI et configuration
- **Analyse des erreurs** : Identifier les sources d'erreurs par batch/dataset

Une particularité de SCRBenchmark est la gestion des données d'entrée pour les modèles probabilistes (scVI, scDeepCluster, scMUG, scCDCG, scNAME). Ces modèles sont conçus pour modéliser des distributions de comptage (Negative Binomial ou ZINB) ou bénéficient de l'apprentissage sur données brutes. Le logiciel préserve donc les comptes bruts des cellules sélectionnées et des gènes HVG dans `adata.layers['original_X']` (ou `adata.raw`) et les injecte spécifiquement dans ces modèles. **Note Importante** : Le mode « Raw Counts » du logiciel utilise toujours la version **filtrée** au niveau des cellules (après QC) et restreinte aux mêmes gènes (HVG) que le mode « Processed ». Cela garantit que les métriques (NMI, ARI) restent comparables car calculées sur le même ensemble de cellules, et évite les erreurs d'indices liées à l'utilisation de matrices de tailles différentes. L'usage de données 100% non-filtrées est proscrit dans le benchmark rigoureux.

2 Métriques d'Évaluation

Le logiciel calcule un ensemble de métriques pour évaluer la qualité du clustering. Le socle principal repose sur NMI, ARI, ACC et Silhouette, complété par UCA, F1_Macro, BalancedACC, RareACC, KNN Purity et NNO selon les données disponibles.

2.1 Normalized Mutual Information (NMI)

La NMI mesure la quantité d'information partagée entre les labels prédits Y et les labels vrais C , normalisée pour être dans $[0, 1]$ [?].

$$\text{NMI}(Y, C) = \frac{2 \cdot I(Y; C)}{H(Y) + H(C)} \quad (1)$$

où :

- $I(Y; C)$ est l'information mutuelle : $I(Y; C) = \sum_{y \in Y} \sum_{c \in C} p(y, c) \log \frac{p(y, c)}{p(y) \cdot p(c)}$
- $H(Y)$ et $H(C)$ sont les entropies respectives

Implémentation

```

1 from sklearn.metrics import normalized_mutual_info_score
2
3 def compute_nmi(labels_true, labels_pred):
4     return normalized_mutual_info_score(
5         labels_true,
6         labels_pred
7     ) # Uses sklearn default (arithmetic mean)

```

2.2 Adjusted Rand Index (ARI)

L'ARI mesure la similarité entre deux clusterings en comptant les paires d'éléments qui sont dans le même cluster ou dans des clusters différents [?].

$$\text{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}} \quad (2)$$

où n_{ij} est le nombre d'objets dans le cluster i de Y et le cluster j de C , $a_i = \sum_j n_{ij}$ et $b_j = \sum_i n_{ij}$.

Propriétés

- ARI = 1 : clustering parfait
- ARI = 0 : clustering aléatoire (espérance)
- ARI < 0 : clustering pire que l'aléatoire

2.3 Clustering Accuracy (ACC) et Stratégie d'Alignement

L'ACC mesure la proportion de cellules correctement classifiées après avoir aligné les clusters prédits avec les labels réels à l'aide de l'algorithme hongrois [?]. Contrairement à la classification supervisée, les clusters n'ont pas de labels intrinsèques (ex : « Cluster 0 » ne veut rien dire). Il faut donc trouver le meilleur mapping (permutation) entre les clusters $\{c\}$ et les labels $\{y\}$.

2.3.1 Le Défi de l'Évaluation par Batch

Lors de l'évaluation des performances sur des sous-groupes (ex : par batch expérimental), deux stratégies existent pour calculer ce mapping, avec un impact majeur sur les scores.

1. L'Ancienne Approche : Alignement Local (Biaisée) Cette méthode calcule un mapping optimal *indépendant* pour chaque batch. Elle considère chaque batch comme un univers isolé.

$$\text{ACC}_{\text{local}}(\text{batch}_i) = \max_{\pi_i \in \mathcal{S}_k} \left(\frac{\sum_{x \in \text{Batch}_i} \mathbb{I}(y(x) = \pi_i(c(x)))}{|\text{Batch}_i|} \right) \quad (3)$$

Problème (Biais Optimiste) : Le modèle peut être incohérent sans être pénalisé. Par exemple, il peut identifier le Cluster 0 comme « Cell Type A » dans le Batch 1, et comme « Cell Type B » dans le Batch 2. L'alignement local s'adapte à cette incohérence et renvoie un score parfait, masquant le manque de robustesse du modèle.

2. La Solution SCRBenchmark : Une Double Approche SCRBenchmark reconnaît l'utilité des deux perspectives et implémente une dualité explicite :

- **Mode Diagnostic (Alignement Local)** : Permet d'identifier des erreurs « purement locales ». Si l'ACC locale est bonne mais l'ACC globale est mauvaise sur un *batch*, cela indique un problème de *consistance* (le modèle « labelle » bien mais change de dictionnaire) plutôt qu'un problème de séparation. Ce mode est activé par défaut dans les utilitaires d'inspection (`src/scrbenchmark/utils`).
- **Mode Benchmark (Alignement Global)** : C'est la métrique officielle rapportée dans ce document. Pour mesurer la véritable capacité de généralisation, nous imposons un **mapping unique** calculé sur l'ensemble du dataset (tous batches confondus). Seule cette méthode respecte les standards du Deep Clustering [?] pour l'évaluation de modèles non-supervisés.

$$\pi_{global} = \arg \max_{\pi \in \mathcal{S}_k} \sum_{x \in D_{total}} \mathbb{K}(y(x) = \pi(c(x))) \quad (4)$$

Interprétation : On cherche la permutation optimale unique π_{global} qui maximise les correspondances sur l'ensemble du dataset (D_{total}).

$$ACC_{global}(batch_i) = \frac{\sum_{x \in Batch_i} \mathbb{K}(y(x) = \pi_{global}(c(x)))}{|Batch_i|} \quad (5)$$

Interprétation : On applique cette permutation unique π_{global} (fixée à l'étape précédente) pour évaluer chaque batch individuellement, interdisant tout ré-alignement opportuniste.

Conclusion : Les chiffres présentés dans les sections suivantes utilisent strictement l'Alignement Global. Si le modèle change la « signification » d'un cluster d'un batch à l'autre, cette incohérence sera sanctionnée.

Alignement pour la Visualisation

Cette logique d'alignement est également utilisée pour la **visualisation**. Les clusters numériques (ex : "0", "1") sont traduits en noms de types cellulaires (ex : "B-Cell", "T-Cell") dans les légendes des graphiques UMAP, en utilisant le mapping calculé sur l'ensemble des données. Cela facilite la comparaison visuelle entre la vérité terrain et les prédictions.

Implémentation

```

1  from scipy.optimize import linear_sum_assignment
2
3  def cluster_acc(y_true, y_pred):
4      # Construire la matrice de confusion
5      D = max(y_pred.max(), y_true.max()) + 1
6      w = np.zeros((D, D), dtype=np.int64)
7      for i in range(len(y_pred)):
8          w[y_pred[i], y_true[i]] += 1
9
10     # Algorithme hongrois (maximisation)
11     row_ind, col_ind = linear_sum_assignment(w.max() - w)
12
13     # Note: Dans SCRBenchmark, pour les métriques par batch,
14     # cet alignement n'est PAS recalculé par batch.
15     # On utilise le mapping global.
16     return w[row_ind, col_ind].sum() / len(y_pred)

```

2.4 Macro F1-Score

Le F1-Score est la moyenne harmonique de la précision et du rappel. Pour gérer les déséquilibres de classes (types cellulaires rares vs abondants), nous utilisons la version **Macro-averaged**.

$$\text{Macro F1} = \frac{1}{|C|} \sum_{c \in C} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \quad (6)$$

où P_c (Précision) et R_c (Rappel) sont calculés pour chaque classe c après alignement optimal (global) des clusters prédits avec les labels réels. Cette métrique traite chaque classe avec la même importance, quelle que soit sa taille. Un modèle qui ignore un type cellulaire rare verra son score Macro F1 chuter drastiquement, contrairement à l'ACC ou au NMI qui sont souvent dominés par les classes majoritaires.

2.5 Balanced Accuracy (B-ACC)

La Balanced Accuracy est la moyenne arithmétique du Rappel de chaque classe. Elle est adaptée aux datasets déséquilibrés.

$$\text{Balanced ACC} = \frac{1}{|C|} \sum_{c \in C} R_c = \frac{1}{|C|} \sum_{c \in C} \frac{TP_c}{TP_c + FN_c} \quad (7)$$

Contrairement à l'Accuracy standard qui mesure le taux de succès global ($\frac{\sum TP}{N}$), la Balanced Accuracy pénalise fortement les erreurs sur les petites classes. Par exemple, si une classe rare (1% des données) est totalement manquée, la B-ACC ne pourra pas dépasser 99% (et sera probablement bien inférieure selon le nombre de classes), signalant immédiatement le défaut de généralisation.

Note sur l'Alignement

Pour le F1-Score et la Balanced Accuracy, nous utilisons la même permutation optimale globale π_{global} que celle calculée pour l'ACC. Cela garantit que toutes les métriques évaluent la même configuration de labels, évitant les incohérences.

2.6 Métrique Spécifique aux Classes Rares : Rare Class Accuracy (Rare-ACC)

Cette métrique « sur-mesure » isole les performances sur les populations minoritaires, souvent noyées dans les métriques globales.

$$\text{Rare-ACC} = \frac{\sum_{x \in D_{rare}} \mathbb{1}(y(x) = \pi_{global}(c(x)))}{|D_{rare}|} \quad (8)$$

où $D_{rare} = \{x \mid \text{freq}(y(x)) < 5\%\}$ est l'ensemble des cellules appartenant à des types cellulaires représentant moins de 5% du jeu de données total. Cette mesure est binaire et impitoyable : elle révèle si l'algorithme est capable de distinguer des signaux faibles (populations rares) ou s'il les assimile systématiquement aux clusters majoritaires (phénomène d'absorption). Elle utilise le même alignement global π_{global} que l'ACC standard.

2.7 Silhouette Score

Le score de silhouette mesure la cohésion intra-cluster et la séparation inter-cluster.

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (9)$$

où :

- $a(i)$: distance moyenne de i aux autres points de son cluster

- $b(i)$: distance moyenne minimale de i aux points des autres clusters

2.8 Nombre de Paramètres (Taille du Modèle)

Pour les algorithmes basés sur le Deep Learning, SCRBenchmark rapporte désormais le nombre total de paramètres entraînaables (N_θ). Cette métrique permet d'évaluer l'efficacité computationnelle et le risque de sur-apprentissage (*overfitting*) des modèles.

Implémentation

```
1 def get_num_parameters(self):
2     if self.model is None:
3         return None
4     return sum(p.numel() for p in self.model.parameters() if p.requires_grad)
```

Le score global est : $S = \frac{1}{n} \sum_{i=1}^n s(i) \in [-1, 1]$

2.9 Filtrage Automatique des Labels Noise

Certains algorithmes de clustering (HDBSCAN, DBSCAN, certaines méthodes basées sur graphes) peuvent assigner des cellules à un label spécial **-1** ou **'noise'** pour indiquer qu'elles n'appartiennent à aucun cluster défini.

Problème : Inclure ces cellules « *noise* » dans le calcul des métriques biaise les résultats car :

- Elles ne sont pas réellement assignées à un cluster
- La comparaison avec les labels ground truth n'a pas de sens pour ces points

Solution SCRBenchmark : Le logiciel filtre automatiquement les échantillons marqués comme « *noise* » **avant** le calcul des métriques.

Labels Noise Reconnus

- Valeurs numériques : -1
- Valeurs textuelles : 'noise', 'Noise', 'NOISE', 'unassigned', 'Unassigned', '-1'

Comportement :

- Si `labels_true[i]` OU `labels_pred[i]` est un label noise $\rightarrow$ cellule exclue
- Le nombre de cellules filtrées est reporté dans `metrics['n_noise_filtered']`
- Le nombre de cellules effectivement évaluées est dans `metrics['n_samples_evaluated']`

Implémentation

```
1 NOISE_LABELS = {-1, '-1', 'noise', 'Noise', 'unassigned'}
2
3 def _filter_noise_samples(labels_true, labels_pred, data=None):
4     mask_true = ~np.isin(labels_true, list(NOISE_LABELS))
5     mask_pred = ~np.isin(labels_pred, list(NOISE_LABELS))
6     valid_mask = mask_true & mask_pred
7     return labels_true[valid_mask], labels_pred[valid_mask], ...
```

2.10 Métriques de Préservation de Structure

Ces métriques évaluent la fidélité de la représentation latente par rapport à la topologie locale des données d'origine et la cohérence des voisinages.

2.10.1 KNN Purity

La pureté KNN mesure la proportion de voisins d'une cellule appartenant au même type cellulaire qu'elle. C'est un indicateur local de la qualité du clustering.

$$\text{KNN Purity} = \frac{1}{N} \sum_{i=1}^N \frac{|\mathcal{N}_k(i) \cap C(c_i)|}{k} \quad (10)$$

où $\mathcal{N}_k(i)$ est l'ensemble des k plus proches voisins de la cellule i dans l'espace latent, et $C(c_i)$ est l'ensemble des cellules du même type que i .

2.10.2 Nearest Neighbor Overlap (NNO)

Le NNO quantifie le taux de recouvrement entre le graphe des k -plus proches voisins calculé sur les données d'origine (espace des gènes ou PCA) et celui calculé sur l'embedding latent. Une valeur élevée (proche de 1.0) indique que l'algorithme a préservé la structure locale des données et n'a pas introduit de distorsions topologiques majeures.

2.10.3 Jaccard Index (Global)

Similaire au NNO mais plus global, cet index calcule la similarité de Jaccard moyenne entre les matrices d'adjacence des k -NN des PCAs par batch et de l'embedding final. Il évalue la conservation des relations cellules-cellules intra-batch.

2.11 Métriques d'Intégration (Suite scIB)

Pour évaluer spécifiquement la qualité de la correction de batch (suppression des effets techniques tout en préservant la biologie), SCRBenchmark intègre les métriques standardisées du benchmark *scIB* (Single-cell Integration Benchmark) [?]. Dans l'implémentation, ces métriques sont calculées uniquement si la dépendance optionnelle `scib_metrics` est installée; sinon elles sont ignorées proprement avec un avertissement.

2.11.1 Notations communes

On note N cellules, B batches, C types cellulaires, b_i le batch de la cellule i , y_i son type cellulaire, et z_i son embedding latent. n_b désigne le nombre de cellules du batch b . $\mathcal{N}_k(i)$ désigne le voisinage k -NN de i dans l'espace latent. Les métriques ci-dessous correspondent à celles agrégées dans `external/original_code/scIB-E/src/scIB-E/main_helpers.py` et dans notre wrapper `src/scrbenchmark/utils/metrics.py`. Dans SCRBenchmark, l'appel `Benchmarker.get_results(min_max_sca` conserve l'échelle native de `scib_metrics` (pas de re-minmax global inter-méthodes).

2.11.2 Métriques de Mélange (Batch Correction)

1) **Silhouette batch** (aussi notée ASW/BRAS selon la version de `scib_metrics`) :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \quad a(i) = \text{dist. moyenne intra-batch}, \quad b(i) = \min_{\beta \neq b_i} \text{dist. moyenne vers batch } \beta \quad (11)$$

La forme utilisée pour la correction de batch est transformée pour que **plus grand = meilleur mélange** :

$$\text{SilhouetteBatch}^* = \frac{1}{N} \sum_{i=1}^N (1 - |s(i)|). \quad (12)$$

2) iLISI (integration LISI) :

$$\text{LISI}_i^{\text{batch}} = \frac{1}{\sum_{b=1}^B p_i(b)^2}, \quad p_i(b) = \frac{1}{k} \sum_{j \in \mathcal{N}_k(i)} \mathbf{1}(b_j = b) \quad (13)$$

et (version normalisée usuelle) :

$$\text{iLISI}_{\text{scaled}} = \frac{\text{iLISI} - 1}{B - 1}. \quad (14)$$

3) kBET : test χ^2 local batch-vs-global.

$$\chi_i^2 = \sum_{b=1}^B \frac{(o_{ib} - e_b)^2}{e_b}, \quad o_{ib} = \frac{1}{k} \sum_{j \in \mathcal{N}_k(i)} \mathbf{1}(b_j = b), \quad e_b = \frac{n_b}{N} \quad (15)$$

$$\text{kBET} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}(p_i \geq \alpha) \quad (16)$$

(p_i : p-valeur du test). Cette écriture est l'*acceptance rate* (équivalente à $1 - \text{rejection rate}$).

4) Graph connectivity : pour chaque type cellulaire c , on construit le sous-graphe KNN $G_c = (V_c, E_c)$ induit par les cellules de type c .

$$\text{GC} = \frac{1}{C} \sum_{c=1}^C \frac{|\text{LCC}(G_c)|}{|V_c|} \quad (17)$$

où $\text{LCC}(G_c)$ est la plus grande composante connexe de G_c .

2.11.3 Métriques de Conservation Biologique (Inter-types)

1) KMeans NMI / KMeans ARI :

$$\hat{y} = \text{KMeans}_k(Z), \quad k = |C|, \quad \text{KMeans NMI} = \text{NMI}(\hat{y}, y), \quad \text{KMeans ARI} = \text{ARI}(\hat{y}, y) \quad (18)$$

(NMI/ARI sont définies en section précédente; ici $y = (y_1, \dots, y_N)$ est le vecteur des labels biologiques de référence).

2) Silhouette label :

$$\text{SilhouetteLabel}^* = \frac{\frac{1}{N} \sum_{i=1}^N s_{\text{label}}(i) + 1}{2}, \quad s_{\text{label}}(i) = \frac{b_{\text{label}}(i) - a_{\text{label}}(i)}{\max(a_{\text{label}}(i), b_{\text{label}}(i))} \quad (19)$$

(plus grand = meilleure séparation biologique).

3) cLISI (cell-type LISI) :

$$\text{LISI}_i^{\text{cell}} = \frac{1}{\sum_{c=1}^C q_i(c)^2}, \quad q_i(c) = \frac{1}{k} \sum_{j \in \mathcal{N}_k(i)} \mathbf{1}(y_j = c) \quad (20)$$

et (version normalisée orientée *higher is better*) :

$$\text{cLISI}_{\text{scaled}} = 1 - \frac{\text{cLISI} - 1}{C - 1}. \quad (21)$$

4) **Isolated labels** : score ASW spécifique aux types rares/batch-spécifiques. En notant $\mathcal{C}_{iso}$ l'ensemble des types isolés et $\mathcal{B}(c)$ les batches contenant le type c :

$$\text{IsolatedLabels} = \frac{1}{|\mathcal{C}_{iso}|} \sum_{c \in \mathcal{C}_{iso}} \frac{\frac{1}{|\mathcal{B}(c)|} \sum_{b \in \mathcal{B}(c)} \text{ASW}_{c,b}}{1 + \log_2 |\mathcal{B}(c)|}. \quad (22)$$

où $\text{ASW}_{c,b}$ est le score de silhouette moyen des cellules du type c dans le batch b .

2.11.4 Métriques de Conservation Intra-types

1) PCR comparison :

$$K = \text{nombre de composantes principales utilisées dans la PCR} \quad (23)$$

$$\text{Var}_{\text{batch}} = \frac{\sum_{j=1}^K \text{Var}(PC_j) R_j^2}{\sum_{j=1}^K \text{Var}(PC_j)} \quad (24)$$

où R_j^2 est le coefficient de détermination de la régression de PC_j sur la variable batch. Le score scIB-E est :

$$\text{PCR}_{\text{batch}} = 1 - \exp\left(-\left|\text{Var}_{\text{batch}}^{\text{pre}} - \text{Var}_{\text{batch}}^{\text{post}}\right|\right). \quad (25)$$

2) **Jaccard index** (ajouté explicitement dans `main_helpers.py`) : pour chaque batch b , on construit deux graphes kNN non orientés G_b^{PCA} (PCA par batch) et G_b^Z (embedding latent), puis

$$J_b = \frac{|E_b^{\text{PCA}} \cap E_b^Z|}{|E_b^{\text{PCA}} \cup E_b^Z|}, \quad \text{Jaccard index} = \frac{1}{B} \sum_{b=1}^B J_b. \quad (26)$$

2.11.5 Score Global (scIB-E)

Pour synthétiser la qualité de l'intégration, un score global pondéré est calculé selon la méthodologie scIB-E :

$$S_{\text{total}} = 0.2 \cdot S_{\text{batch}} + 0.4 \cdot S_{\text{inter}} + 0.4 \cdot S_{\text{intra}} \quad (27)$$

où :

- S_{batch} (Correction de Batch) : moyenne arithmétique de {Silhouette batch, iLISI, KBET, Graph connectivity}.
- S_{inter} (Conservation Inter-Types) : moyenne arithmétique de {Isolated labels, KMeans NMI, KMeans ARI, Silhouette label, cLISI}.
- S_{intra} (Conservation Intra-Types) : moyenne arithmétique de {PCR comparison, Jaccard index}.

Dans le code SCRbenchmark (`compute_scib_metrics`), ces moyennes sont calculées sur les métriques disponibles (non NaN), puis combinées avec les poids 0.2/0.4/0.4.

2.12 Classification vs Clustering : Pourquoi ces métriques ?

Il est légitime de se demander pourquoi nous utilisons des métriques typiques de la **classification supervisée** (Accuracy, F1-Score, Précision, Rappel) pour évaluer des algorithmes de **clustering non-supervisé**.

Distinction Fondamentale :

- **Classification** : Le modèle apprend à prédire des classes prédéfinies (ex : « Cell type A », « Cell type B »). La sortie y_{pred} correspond directement à y_{true} .

- **Clustering** : Le modèle regroupe les données par similarité sans connaître les étiquettes. La sortie est un index de groupe arbitraire (ex : « Cluster 0 », « Cluster 1 ») qui n'a aucun sens intrinsèque.

Le « Pont » de l'Évaluation : Pour utiliser des métriques supervisées dans un benchmark de clustering où la vérité terrain est connue, nous devons **aligner** l'espace des clusters prédits sur l'espace des labels réels. C'est le rôle de l'algorithme hongrois (cité plus haut pour l'ACC).

$$\text{Clustering} \xrightarrow{\text{Alignement Optimal (Mapping)}} \text{Pseudo-Classification} \rightarrow \text{Métriques (ACC, F1, ...)} \quad (28)$$

Une fois ce *mapping* établi (ex : « Cluster 2 » correspond à « Cell Type B »), le résultat peut être traité comme une classification et toutes les métriques standards deviennent calculables. Ces métriques (ACC, F1, etc.) fournissent une information par type cellulaire (ex : « le modèle confond le type A et le type B »), contrairement aux métriques non-supervisées comme le Silhouette Score, qui évaluent la géométrie des clusters sans référence aux labels biologiques.

2.13 Rigueur Statistique et Comparaison Multiple

Lors de comparaisons multi-répétitions (par exemple 5 ou 10 runs avec différentes graines aléatoires), le logiciel effectue automatiquement des tests statistiques pour évaluer si les différences de performance entre algorithmes sont significatives.

- **Test Global** :
 - *Non-paramétrique* (Défaut) : Test de Kruskal-Wallis.
 - *Paramétrique* : ANOVA à un facteur.
- **Test Post-hoc** : Si le test global est significatif ($p < \alpha$), des comparaisons par paires sont effectuées :
 - *Non-paramétrique* : Test de Dunn avec correction de Bonferroni.
 - *Paramétrique* : Test de Tukey HSD.
- **Compact Letter Display (CLD)** : Les résultats sont présentés sous forme de lettres (a, b, c, ab). Deux algorithmes partageant la même lettre ne sont pas statistiquement différents au seuil α choisi (généralement 0,05).

Visualisation des Tests

Les boxplots dans l'interface et les rapports exportés incluent ces lettres de significativité au-dessus de chaque boîte pour une interprétation immédiate des performances.

3 Pipeline de Prétraitement Standardisé

Avant toute analyse, les données brutes de séquençage subissent un traitement rigoureux pour éliminer le bruit technique et biologique. Ce pipeline, implémenté dans `src/scrbenchmark/utils/data_handler`, suit un ordre strict pour garantir l'intégrité scientifique :

1. **Import QC** : Chargement et filtrage qualitatif (Cellules/Gènes).
2. **Dropout (Op.)** : Simulation de dégradation sur les comptes bruts.
3. **Size Factors** : Calcul (ou re-calcul post-dropout) de la profondeur.
4. **Normalization** : Mise à l'échelle (Target Sum) et Log1p (sauf si Batch Correction pré-appliquée).
5. **Feature Selection** : Identification des HVGs (et sauvegarde des données pré-HVG pour scMAE).

6. **Batch Correction (Op.)** : Intégration des données (scVI-DCT ou sysVI) sur l’espace des HVGs.
7. **Scaling** : Centrage-Réduction final (Z-score).

3.1 Filtrage Qualité (Quality Control)

Cette étape élimine les cellules et gènes non informatifs ou de mauvaise qualité.

— **Filtrage des Cellules** :

- **min_genes_per_cell** (défaut : 200) : Élimine les cellules mortes ou les gouttelettes vides ayant capturé peu d’ARN.
- **max_genes_per_cell** (défaut : None, donc désactivé) : Seuil optionnel pour éliminer les doublets potentiels (deux cellules capturées dans la même gouttelette), qui présentent une complexité anormalement élevée.

— **Filtrage des Gènes** :

- **min_cells_per_gene** (défaut : 3) : Élimine les gènes exprimés dans trop peu de cellules pour être statistiquement exploitables. Cela réduit considérablement la dimensionnalité sans perte d’information biologique majeure.

3.2 Simulation de Dropout (Optionnelle)

Si activée, cette étape simule les échecs de capture moléculaire **au niveau des comptes bruts**, avant toute normalisation. Cela permet d’évaluer la robustesse des algorithmes sur des données dégradées, tout en gardant un pipeline cohérent.

Conséquence sur les Facteurs de Taille : Les modèles probabilistes (scVI, scDeepCluster) utilisent des « *Size Factors* » pour modéliser la profondeur de séquençage de chaque cellule (souvent calculés comme le nombre total de comptes par cellule). Dans SCRBenchmark, si le dropout est appliqué, les **Size Factors sont recalculés** sur la matrice dégradée. Cela garantit que le modèle reçoit une information de profondeur cohérente avec les données d’entrée (bruitées), simulant fidèlement une expérience de moindre qualité technique plutôt qu’une simple injection de zéros incohérents.

3.3 Normalisation de la Taille de Librairie

La profondeur de séquençage (nombre total de lectures) varie d’une cellule à l’autre pour des raisons techniques (efficacité de capture, PCR).

$$X_{norm} = X \times \frac{\text{target_sum}}{\sum X_{cell}} \quad (29)$$

Le logiciel utilise un **Target Sum de 10,000** par défaut (paramètre `target_sum`), configurable via l’interface.

3.4 Transformation Logarithmique

Applique $X_{log} = \log(X_{norm} + 1)$. **Rôle** :

- Stabilise la variance (l’expression des gènes suit souvent une loi de puissance).
- Rend la distribution plus « normale », ce qui est requis par de nombreux algorithmes (PCA, certains Autoencodeurs).
- Réduit l’impact des valeurs extrêmes d’expression.

3.5 Optimisation de la Correction de Batch

Le logiciel optimise automatiquement le pipeline lorsque la correction de batch est activée.

Le problème de la double-normalisation : De nombreux algorithmes (scDeepCluster, scMAE, scNAME, scHNTL) effectuent une normalisation interne (ex : scale, log1p) par défaut. Si l'utilisateur fournit des données déjà corrigées par un algorithme comme scVI-DCT ou sysVI, une nouvelle normalisation interne risquerait « d'effacer » ou de biaiser la correction de *batch* effectuée en amont.

Solution SCRBenchmark :

- Le logiciel détecte automatiquement si *Batch Correction* est actif ET si l'utilisateur a choisi des données *Processed* en entrée.
- Dans ce cas, le paramètre `use_raw_data` de l'algorithme est forcé à `False` (via une injection de paramètre dans l'UI et la CLI).
- Cela désactive la **préparation spécifique aux comptes bruts** (normalisation/log interne) pour les algorithmes qui respectent `use_raw_data`. Certains modèles appliquent encore un scaling de stabilité même en mode *Processed* (ex : scDeepCluster), mais l'idée est d'éviter une double normalisation des comptes bruts.

3.6 Sélection des Gènes Hautement Variables (HVG)

Sur les 20,000 gènes humains, la majorité sont du « bruit » ou des gènes de ménage constants.

- **Méthode** : Flavor 'Seurat' (dispersion normalisée par bin de moyenne).
- **Sélection** : Top 2,000 à 5,000 gènes.
- **Rôle** : Concentre l'analyse sur les gènes dont l'expression varie le plus entre les types cellulaires, réduisant la dimensionnalité et le bruit.

3.7 Correction de Batch (Optionnelle)

Figure indisponible : fichier `Correction_Batch.png` non trouvé.

FIGURE 1 – Schéma du pipeline scVI-DCT pour la correction de batch dans SCRBenchmark. Les comptes bruts et l'information de batch alimentent l'encodeur pour apprendre un espace latent nettoyé ($z = 30$). Deux usages sont ensuite possibles : (A) génération d'une matrice corrigée X' (2000 gènes) utilisée comme entrée d'un autre algorithme (par exemple Enhanced AE), ou (B) clustering direct dans l'espace latent (K-Means) pour produire les clusters finaux.

L'hétérogénéité technique entre différents lots d'expériences (*batch effects*) est un défi majeur en scRNA-seq. SCRBenchmark propose désormais deux méthodes principales pour corriger ces effets avant le clustering, intégrant des approches avancées d'apprentissage profond :

- **scVI-DCT** (Basé sur la *Domain Class Triplet loss*) : Cette approche étend le cadre variationnel classique (scVI [?]) en intégrant une fonction de perte spécifique, la *Domain Class Triplet loss*, pour optimiser conjointement la suppression des effets de lot et la conservation biologique.
- **Mécanisme Triplet** : Contrairement aux pertes triplets standard, cette méthode sélectionne pour une cellule ancre (a) :
 1. Un **positif** (p) : une cellule du *même type cellulaire* mais provenant d'un *lot différent*.
 2. Un **négatif** (n) : une cellule du *même lot* mais d'un *type cellulaire différent*.

- **Formulation Mathématique** : L’objectif est de minimiser la distance entre l’ancre et le positif tout en maximisant la distance avec le négatif, selon l’équation :

$$Loss_{DCT} = [D(a, p) - D(a, n) + \tau]_+$$

où D représente la distance euclidienne et τ est une marge de séparation définie.

- **Préservation Intra-Type (Corr-MSE)** : Pour éviter le lissage excessif des sous-populations (over-smoothing), le modèle minimise l’erreur quadratique moyenne entre les matrices de corrélation **gene-gene** de l’entrée et de l’espace latent.

Détail d’implémentation (scIB-E Methodology) : La matrice de corrélation cible n’est pas calculée sur les données globales (ce qui inclurait les effets de batch), mais sur des **PCA calculées séparément par batch**.

$$\mathcal{L}_{Corr} = \frac{1}{B} \sum_{b=1}^B \|\text{Corr}(Z_b) - \text{Corr}(\text{PCA}(X_b))\|_F^2 \quad (30)$$

Ces composantes principales par batch sont passées au modèle scVI via des *continuous covariates* dédiées, garantissant que le modèle apprend à préserver la topologie locale spécifique à chaque condition expérimentale.

- **sysVI** : Variante moderne de VAE conditionnel (Hrovatin et al., 2023) conçue pour les effets batch substantiels. Elle combine un prior de type VampPrior et une perte de cycle-consistency pour améliorer l’alignement inter-systèmes tout en préservant la variabilité biologique.

Ces méthodes sont appliquées **avant le scaling** final, garantissant que le clustering aval (K-Means, Leiden, modèles profonds) opère sur des représentations biologiquement cohérentes.

3.7.1 Stratégies HVG en Mode Benchmark

Le choix des HVGs conditionne le risque de **Data Leakage**. SCRBenchmark propose trois stratégies de sélection (contrôlables via le flag `-hvg-strategy`) :

- **train_only** (Défaut) : Les gènes sont sélectionnés uniquement sur le jeu d’entraînement. C’est la méthode la plus rigoureuse pour garantir l’absence de fuite d’information.
- **all_data** : La sélection est effectuée sur l’intégralité du dataset avant le split (disponible via CLI/config). À n’utiliser que pour des analyses exploratoires simples.
- **per_batch_union** : Sélectionne le top N gènes dans chaque batch individuellement, puis calcule l’union. Cette approche est utile pour préserver des marqueurs rares spécifiques à certains lots expérimentaux.

3.7.2 Protection des Données : Copie Avant Modification

Problème : Une correction de batch qui modifie `adata.X in-place` peut casser l’itération interactive dans Streamlit et empêcher un retour propre dans le workflow.

Solution SCRBenchmark : Toutes les fonctions de correction de batch travaillent sur une copie de l’objet AnnData :

```

1 def apply_batch_correction_safe(adata, method='scvi', params=None):
2     # IMPORTANT: travailler sur une copie
3     adata = adata.copy()
4
5     # Sauvegarder les donnees brutes avant correction
6     if 'original_X' not in adata.layers:
7         adata.layers['original_X'] = adata.X.copy()
```

```

8
9     adata_corr, embeddings = apply_batch_correction(adata, method=
10         method, params=params)
11     ...
12     return adata_corr, embeddings

```

Méthodes protégées : `apply_scvi_batch_correction`, `apply_sysvi_batch_correction`.

3.7.3 Validation des Raw Counts pour scVI

scVI est un modèle génératif basé sur la distribution Negative Binomial, qui **exige des comptes entiers** (raw counts) en entrée.

Validation Stricte des Raw Counts

Si le logiciel détecte que les données fournies à scVI sont normalisées (nombres à virgule flottante) et qu'aucun raw count n'est disponible dans `layers['original_X']` ou `adata.raw`, une **erreur** est levée :

```

1 ValueError: scVI batch correction requires raw counts
2 but only normalized data is available. Please ensure
3 raw counts are preserved in layers['original_X'].

```

Raison scientifique : Appliquer scVI sur des données log-normalisées viole les hypothèses du modèle et produit des résultats non interprétables.

3.8 Mise à l'Échelle (Scaling)

Centre et réduit les données (moyenne = 0, variance = 1), **après la correction de batch**.

$$X_{scaled} = \frac{X_{log} - \mu}{\sigma} \quad (31)$$

Rôle : Assure que tous les gènes contribuent de manière comparable au clustering, empêchant les gènes très exprimés de dominer la fonction de perte (ex : K-Means, PCA). *Note : Les valeurs sont généralement "clippées" (ex : max 10) pour éviter que des outliers n'écrasent le gradient.*

4 Algorithmes de Clustering

Le logiciel implémente un registre d'algorithmes extensible (chargement robuste selon dépendances), incluant des méthodes classiques (PCA+K-Means, PCA+Leiden), des approches Deep Learning (scDeepCluster, scMAE et variantes, scNAME, scHNTL, scMUG, scCDCG, Simple Autoencoder, scVI-DCT, Enhanced Autoencoder, scProto) et des algorithmes optionnels (ex : C2S+Leiden). L'objectif est de fournir des métriques standardisées (NMI, ARI, ACC, Silhouette) ainsi que des métriques de robustesse (UCA, F1_Macro, BalancedACC, RareACC, KNN Purity, NNO), pour évaluer finement les modèles face à la complexité biologique et technique (*batch effects*).

4.1 Méthodes Classiques

4.1.1 PCA + K-Means

Réduction de dimension par ACP suivie d'un clustering K-Means.

Fonction objectif K-Means (Lloyd, 1982) :

$$\mathcal{L}_{\text{K-Means}} = \sum_{i=1}^n \min_{j \in \{1, \dots, k\}} \|z_i - \mu_j\|^2 = \sum_{j=1}^k \sum_{i \in C_j} \|z_i - \mu_j\|^2 \quad (32)$$

où $\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} z_i$ est le centroïde du cluster C_j .

Hyperparamètres sensibles – PCA + K-Means

Paramètre	Défaut	Criticité	Description
n_clusters (k)	8	Critique	Nombre de clusters cible. Doit correspondre au nombre réel de types cellulaires.
use_ground_truth_k	True	Critique	Mode Oracle : utilise k des labels. À éviter en benchmark !
n_pca_components	0	Important	Dimensions PCA. 0 = auto (Cattell's scree test). Trop peu = perte d'information, trop = bruit.
kmeans_n_init	20	Modéré	Nombre d'initialisations K-Means. Plus = meilleure convergence mais plus lent.

Mode d'exécution : Inductif. Baseline simple supportant totalement le *transfer learning* (projection PCA + prédiction K-Means aux centroïdes).

4.1.2 PCA + Leiden

L'algorithme Leiden [?] optimise la modularité sur un graphe KNN.

Fonction de modularité :

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (33)$$

où :

- A_{ij} : adjacence (poids de l'arête entre i et j)
- $k_i = \sum_j A_{ij}$: degré du nœud i
- $m = \frac{1}{2} \sum_{i,j} A_{ij}$: nombre total d'arêtes
- γ : paramètre de résolution

Hyperparamètres sensibles – PCA + Leiden

Paramètre	Défaut	Criticité	Description
leiden_resolution (γ)	0.2	Critique	Résolution Leiden. $\gamma \uparrow$ = plus de clusters, $\gamma \downarrow$ = moins.
leiden_neighbors	19	Important	Nombre de voisins pour le graphe KNN.
n_pca_components	0	Modéré	Dimension de l'espace pour le graphe de voisins. 0 = auto.

Mode d'exécution : Pseudo-Inductif. Bien que Leiden soit basé sur graphe (transductif), l'implémentation utilise un classifieur KNN sur les *embeddings* PCA pour prédire les labels des nouvelles données (*transfer learning* supporté).

4.2 Méthodes Deep Learning

4.2.1 scDeepCluster

Référence : Tian et al. [?], « Clustering single-cell RNA-seq data with a model-based deep learning approach », Nature Machine Intelligence, 2019.

Combine un auto-encodeur avec perte ZINB et un clustering DEC (Deep Embedded Clustering).

Architecture :

$$x \xrightarrow{\text{Encoder}} z \in \mathbb{R}^d \xrightarrow{\text{Decoder}} (\mu, \theta, \pi) \quad (34)$$

Fonction de perte totale :

$$\mathcal{L} = \mathcal{L}_{\text{ZINB}} + \gamma \cdot \mathcal{L}_{\text{cluster}} \quad (35)$$

avec $\mathcal{L}_{\text{cluster}} = \gamma \cdot \mathcal{L}_{\text{KL,raw}}$ dans le code (`cluster_loss()` applique déjà un facteur γ , puis la boucle d'entraînement remultiplie par γ). Ainsi, le poids effectif du terme KL est γ^2 .

1. Perte de reconstruction ZINB (Zero-Inflated Negative Binomial) :

$$\mathcal{L}_{\text{ZINB}} = -\frac{1}{nG} \sum_{i=1}^n \sum_{g=1}^G \log P(x_{ig} \mid \mu_{ig}, \theta_{ig}, \pi_{ig}, s_i) \quad (36)$$

avec la distribution ZINB :

$$P(x \mid \mu, \theta, \pi) = \pi \delta_0(x) + (1 - \pi) \text{NB}(x \mid \mu, \theta) \quad (37)$$

où :

- μ : moyenne prédite (sortie du décodeur, activation exponentielle clampée)
- θ : dispersion, aussi appelée *inverse overdispersion* (activation softplus clampée)
- π : probabilité de dropout (sortie sigmoïde)
- s : *size factor* (facteur de normalisation par cellule)

Explication détaillée de la perte ZINB. La distribution ZINB modélise les comptes d'expression génique en combinant deux phénomènes biologiques et techniques distincts :

Pourquoi la ZINB ? Les données scRNA-seq présentent deux caractéristiques que la MSE classique ne capture pas adéquatement :

1. **Excès de zéros (*zero-inflation*)**. Une grande proportion des comptes sont nuls. Ces zéros ont deux origines : (a) le gène n'est réellement pas exprimé dans la cellule, ou (b) le gène est exprimé mais le transcrit n'a pas été capturé par le séquençage (*dropout technique*).
2. **Surdispersion**. Quand un gène est exprimé, la variance des comptes dépasse largement la moyenne (la distribution de Poisson est insuffisante), ce qui est typique des données de comptage biologique.

Construction de la distribution. La ZINB est un modèle à mélange (*mixture model*) entre :

- un **point de masse en zéro** (avec probabilité π) : représente les dropouts techniques,
- une **distribution Binomiale Négative** (avec probabilité $1 - \pi$) : modélise les comptes non nuls avec surdispersion.

Pour un gène g d'une cellule donnée, la vraisemblance s'écrit :

$$P(x_g \mid \mu_g, \theta_g, \pi_g) = \begin{cases} \pi_g + (1 - \pi_g) \cdot \text{NB}(0 \mid \mu_g, \theta_g) & \text{si } x_g = 0 \\ (1 - \pi_g) \cdot \text{NB}(x_g \mid \mu_g, \theta_g) & \text{si } x_g > 0 \end{cases} \quad (38)$$

où la composante Binomiale Négative est :

$$\text{NB}(x \mid \mu, \theta) = \frac{\Gamma(x + \theta)}{\Gamma(\theta) \cdot x!} \cdot \left(\frac{\theta}{\theta + \mu} \right)^\theta \cdot \left(\frac{\mu}{\theta + \mu} \right)^x \quad (39)$$

Rôle de chaque paramètre prédit par le décodeur :

Param.	Activation	Domaine	Signification biologique
μ_g	exp(.) clampé	$[10^{-5}, 10^6]$	Niveau d'expression moyen attendu pour le gène g , multiplié par le <i>size factor</i> s de la cellule
θ_g	softplus clampé	$[10^{-4}, 10^4]$	Paramètre de dispersion : θ petit = forte variance ; $\theta \rightarrow \infty$ = convergence vers Poisson
π_g	sigmoïde	$[0, 1]$	Probabilité que le zéro observé soit un dropout technique plutôt qu'une absence réelle d'expression

Implémentation dans le code. La perte ZINB est calculée en log-vraisemblance négative, avec deux branches selon que $x_g \approx 0$ ou $x_g > 0$:

```

1  def forward(self, x, mean, disp, pi, scale_factor):
2      mean = mean * scale_factor[:, None]          # mu * size_factor
3
4      # Log-vraisemblance negative de la composante NB
5      t1 = lgamma(disp) + lgamma(x + 1) - lgamma(x + disp)
6      t2 = (disp + x) * log(1 + mean/(disp)) + x * (log(disp) - log(
7          mean))
8      nb_final = t1 + t2                            # -log NB(x | mu, theta)
9
10     # Cas x > 0 : seule la composante NB contribue
11     nb_case = nb_final - log(1 - pi)
12
13     # Cas x = 0 : melange dropout + NB(0)
14     zero_nb = (disp / (disp + mean)) ** disp      # NB(0 | mu, theta)
15     zero_case = -log(pi + (1 - pi) * zero_nb)
16
17     # Selection automatique selon la valeur de x
18     result = where(x <= 1e-8, zero_case, nb_case)
19     return torch.mean(result) # moyenne sur tous les genes et
                                cellules

```

Listing 1 – Perte ZINB (classe ZINBLoss)

Avantages de la ZINB par rapport à la MSE.

- **Modélisation explicite des dropouts** : le paramètre π permet de distinguer un zéro biologique (gène non exprimé) d'un zéro technique (dropout), ce que la MSE traite indifféremment.
- **Adaptation à la surdispersion** : le paramètre θ ajuste la variance indépendamment de la moyenne, capturant l'hétérogénéité biologique inter-cellulaire.
- **Opération sur comptes bruts** : la ZINB travaille directement sur les comptes de lecture non normalisés (à l'aide du *size factor*), évitant les biais introduits par la normalisation logarithmique.
- **Trois sorties spécialisées** : chaque gène reçoit son propre triplet (μ_g, θ_g, π_g) , permettant une modélisation fine gène par gène.

2. Terme de clustering KL (DEC) :

$$\mathcal{L}_{\text{KL,raw}} = \text{KL}(P\|Q) = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (40)$$

Soft Assignment (Student-t, forme implémentée) :

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2/\alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'=1}^k (1 + \|z_i - \mu_{j'}\|^2/\alpha)^{-\frac{\alpha+1}{2}}} \quad (41)$$

Target Distribution P (sharpened Q) :

$$p_{ij} = \frac{q_{ij}^2/f_j}{\sum_{j'} q_{ij'}^2/f_{j'}} \quad \text{où} \quad f_j = \sum_i q_{ij} \quad (42)$$

Dans ces trois équations (bloc DEC), les notations sont :

- n : nombre total de cellules d’entraînement ;
- k : nombre de clusters ;
- $z_i \in \mathbb{R}^d$: embedding latent de la cellule i ;
- $\mu_j \in \mathbb{R}^d$: centroïde latent du cluster j ;
- q_{ij} : probabilité d’appartenance souple de la cellule i au cluster j ;
- α : paramètre de la Student-t dans l’assignation souple (dans ce code, $\alpha = 1$ par défaut) ;
- p_{ij} : distribution cible “affûtée” construite à partir de q_{ij} ;
- f_j : masse totale assignée au cluster j , utilisée pour corriger l’effet des grands clusters.

Le terme utilisé à l’entraînement est :

$$\mathcal{L}_{\text{cluster}} = \gamma \cdot \mathcal{L}_{\text{KL,raw}}, \quad \mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ZINB}} + \gamma \cdot \mathcal{L}_{\text{cluster}} \quad (43)$$

soit $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ZINB}} + \gamma^2 \mathcal{L}_{\text{KL,raw}}$ dans cette implémentation.

Hyperparamètres sensibles – scDeepCluster

Paramètre	Défaut	Criticité	Description
n_clusters	0	Critique	Nombre de clusters. 0 = auto-estimation via Leiden.
use_ground_truth_k	True	Critique	Mode Oracle : utilise k des labels. À éviter en benchmark !
gamma (γ)	1.0	Critique	Balance reconstruction/clustering. $\gamma \uparrow$ = clustering plus influent. Poids effectif = γ^2 .
pretrain_epochs	200	Important	Initialisation des représentations. Trop peu = mauvaise initialisation.
z_dim	32	Important	Capacité de l'espace latent (16–64 recommandé).
force_float32	False	Modéré	Forcer float32 pour reproductibilité cross-platform (MPS/CUDA/CPU).
learning_rate	0.001	Modéré	Taux d'apprentissage. Affecte la stabilité de convergence.

Mode d'exécution : Inductif.

- *Training* : Optimisation conjointe ZINB + KL divergence.
- *Prediction* : $y = \arg \max_j q_{ij}$. L'encodeur produit z , puis la distribution *soft assignment* q (Student-t) est calculée par rapport aux centroïdes appris.

Note sur les données : scDeepCluster attend en entrée les **comptes bruts** des gènes déjà sélectionnés (HVGs). Il applique sa propre normalisation/scaling interne.

Adaptations pour le Benchmarking Inductif (Spécifique SCRBenchmark)

- Capacité d'Inférence (*Transfer Learning*)** : Ajout d'une méthode `predict()` pour projeter de nouvelles cellules dans l'espace latent appris.
 - *Projection* : $z_{new} = \text{Encoder}(x_{new})$
 - *Assignment* : Distribution q_{ij} (Student-t) entre z_{new} et les centroïdes μ_j appris.
 - *Label* : $y_{pred} = \arg \max_j q_{ij}$
- Reproductibilité Cross-Platform (`force_float32`)** : Option forçant le pipeline en float32 pour garantir des résultats identiques entre CPU/CUDA et Apple Silicon (MPS).

4.2.2 scMAE (Masked Autoencoder)

Référence : Fang et al. [?], « scMAE : a masked autoencoder for single-cell RNA-seq clustering », Bioinformatics, 2024.

Principe : Perturber l'expression génique en mélangeant aléatoirement certaines valeurs avec des valeurs provenant d'autres cellules (shuffling), puis utiliser un auto-encodeur masqué pour reconstruire les données originales et prédire quelles valeurs ont été perturbées.

Fonction de perte totale :

$$\mathcal{L} = (1 - \gamma)\mathcal{L}_r + \gamma\mathcal{L}_m \quad (44)$$

1. Génération des données masquées :

La matrice d'expression masquée X^M est obtenue par :

$$X_{ij}^M = X_{ij} \cdot (1 - M_{ij}) + X'_{ij} \cdot M_{ij} \quad (45)$$

où X' est la matrice avec les valeurs mélangées (shuffled) par colonne, et $M_{ij} \sim \text{Bernoulli}(p_j)$ est le masque binaire. À l'entraînement, l'implémentation utilise ensuite le masque effectif $M_{ij}^* = \mathbb{I}[X_{ij}^M \neq X_{ij}]$ (positions réellement modifiées après shuffle).

2. Perte de reconstruction pondérée :

$$\mathcal{L}_r = \frac{1}{n \cdot p} \sum_{i,j} \omega_{ij} \cdot |X_{ij} - \hat{X}_{ij}|^2 \quad (46)$$

où les poids $\omega_{ij} = M_{ij}^* \cdot \kappa + (1 - M_{ij}^*) \cdot (1 - \kappa)$ permettent de pondérer différemment les entrées masquées et non-masquées, avec κ un hyperparamètre.

3. Perte d'estimation du masque (cross-entropy) :

$$\mathcal{L}_m = -\frac{1}{n \cdot p} \sum_{i,j} \left[M_{ij}^* \log \hat{M}_{ij} + (1 - M_{ij}^*) \log(1 - \hat{M}_{ij}) \right] \quad (47)$$

où $\hat{M}$ est le masque prédit (dans le script : logits, optimisés via `BCEWithLogitsLoss`). Cette tâche auxiliaire aide le modèle à capturer les corrélations entre gènes.

Notations du bloc scMAE :

- n : nombre de cellules ; p : nombre de gènes (features) ;
- $X \in \mathbb{R}^{n \times p}$: matrice d'expression originale ;
- X' : matrice “shuffle” obtenue en permutant les valeurs gène par gène ;
- $M \in \{0, 1\}^{n \times p}$: masque binaire, avec $M_{ij} \sim \text{Bernoulli}(p_j)$;
- M^* : masque effectif utilisé par la loss, $M_{ij}^* = \mathbb{I}[X_{ij}^M \neq X_{ij}]$;
- p_j : probabilité de masquage du gène j ;
- X^M : entrée corrompue injectée dans l'encodeur ;
- $\hat{X}$: reconstruction prédite ; $\hat{M}$: probabilité prédite qu'une entrée ait été masquée ;
- ω_{ij} : poids local de reconstruction ; κ : balance masqué/non-masqué ;
- γ : poids global entre reconstruction ($\mathcal{L}_r$) et estimation du masque ($\mathcal{L}_m$).

Processus :

1. Générer X' en mélangeant X par colonne et créer le masque M
2. Créer X^M en combinant X et X' selon M
3. Encoder X^M : $Z = E(X^M)$ (architecture : LayerNorm + Mish)
4. Prédire le masque : $\hat{M} = W_M \cdot Z$ (logits, BCE avec sigmoid implicite)
5. Reconstruire : $\hat{X} = D([Z; \hat{M}])$ (décodeur prend la concaténation)
6. Clustering final avec K-means (si $n < 10000$) ou Leiden sur Z

Hyperparamètres sensibles – scMAE

Paramètre	Défaut	Criticité	Description
masking_rate (p)	0.4	Critique	Taux de masquage optimal. La corruption par <i>shuffle</i> est moins agressive que le masquage à zéro.
masked_data_weight (κ)	0.75	Important	Poids relatif des positions masquées vs non-masquées dans la reconstruction.
mask_loss_weight (γ)	0.7	Important	Balance entre reconstruction ($\mathcal{L}_r$) et estimation du masque ($\mathcal{L}_m$).
clustering_method	auto	Modéré	K-Means pour $n < 10\,000$, Leiden sinon.

Mode d'exécution : Inductif.

- *Training* : Auto-encodeur masqué apprenant des *features* robustes z .
- *Prediction* : Projection $z = E(x)$, puis assignation aux centroïdes K-Means appris. Si Leiden a été utilisé, *fallback* sur K-Means de secours.

Note sur les données : scMAE est conçu pour travailler sur les **comptes bruts** avec son propre pipeline de prétraitement (SizeFactors + Log + HVG selection + Scale + SVD imputation). Lorsque `use_own_preprocessing=True` (défaut), l'algorithme applique sa propre sélection de 1 000 gènes HVG.

Adaptations pour le Benchmarking Inductif (Spécifique SCRBenchmark)

Pour adapter scMAE (initialement transductif) au contexte de *benchmark* Train/Test :

- Préservation des Paramètres de Prétraitement** : Durant `fit()`, l'algorithme stocke les paramètres essentiels :
 - `_selected_hvg_genes` : Liste des gènes HVG sélectionnés
 - `_scale_mean` / `_scale_std` : Statistiques de centrage
 - `svd_imputator` : Objet SVD ajusté pour l'imputation
 - `clusterer` : Objet KMeans ajusté (si utilisé)
- Prétraitement Cohérent des Données de Test** : La méthode `_get_preprocessed_test_data()` applique exactement le même pipeline :
 - Récupération des comptes bruts pré-HVG
 - Normalisation par cellule + transformation logarithmique
 - Filtrage aux mêmes gènes HVG que l'entraînement
 - Application de la permutation des gènes pour l'alignement des *features*
 - Scaling avec les statistiques de l'entraînement
 - Imputation SVD via le `svd_imputator` ajusté
- Prédiction Inductive Pure (KMeans)** : $z_{test} = E(X_{test})$, puis $y = \text{clusterer.predict}(z_{test})$. Approche purement inductive : aucun ré-entraînement nécessaire.

Note : Si Leiden est utilisé (rare, $N \geq 10k$), *fallback* sur K-Means de secours sur les *embeddings* projetés.

4.2.3 scNAME

Référence : Wan et al. [?], « scNAME : Neighborhood contrastive clustering with ancillary mask estimation for scRNA-seq data », Bioinformatics, 2022.

Principe : Combiner un auto-encodeur basé sur le modèle ZINB avec une tâche auxiliaire d'estimation de masque pour explorer les corrélations entre gènes, et un apprentissage contrastif de voisinage pour exploiter la similarité cellulaire.

Fonction de perte totale :

$$\mathcal{L} = \mathcal{L}_{\text{ZINB}} + \alpha \mathcal{L}_{\text{mask}} + \beta \mathcal{L}_{\text{neighbor}} + \gamma \mathcal{L}_{\text{soft}} \quad (48)$$

1. Perte de reconstruction ZINB (similaire à scDeepCluster) :

$$\mathcal{L}_{\text{ZINB}} = \frac{1}{n \cdot p} \sum_{i=1}^n \sum_{j=1}^p \left[-\log P(X_{ij}^c \mid \hat{\pi}_{ij}, \hat{\mu}_{ij}, \hat{\theta}_{ij}) + \lambda_r \hat{\pi}_{ij}^2 \right] \quad (49)$$

avec λ_r le coefficient de ridge sur π ($\lambda_r = 1$ dans le script via `ZINBLoss(ridge_lambda=1.0)`).

2. Génération des données corrompues :

La matrice corrompue $\tilde{X}$ est créée par :

$$\tilde{X} = M \odot X' + (1 - M) \odot X \quad (50)$$

où X' est obtenu en mélangeant aléatoirement X par colonne, M est un masque binaire avec $M_{ij} \sim \text{Bernoulli}(p_{\text{mask}})$, et $\odot$ est le produit élément par élément. Le code utilise ensuite $M_{ij}^* = \mathbb{I}[\tilde{X}_{ij} \neq X_{ij}]$ comme cible de la tâche de masque.

3. Perte d'estimation du masque (cross-entropy) :

$$\mathcal{L}_{\text{mask}} = -\frac{1}{n \cdot p} \sum_{i,j} \left[M_{ij}^* \log \hat{M}_{ij} + (1 - M_{ij}^*) \log(1 - \hat{M}_{ij}) \right] \quad (51)$$

Cette tâche force le réseau à apprendre les dépendances entre gènes pour identifier les valeurs corrompues.

4. Perte de clustering soft K-means pondéré :

$$\mathcal{L}_{\text{soft}} = \frac{1}{n} \sum_{i=1}^n \sum_{q=1}^K \kappa_{iq} \|z_i - \phi_q\|^2 \quad (52)$$

où κ_{iq} est un poids Gaussien normalisé avec inflation :

$$\kappa_{iq} = \frac{(\kappa'_{iq})^2}{\sum_s (\kappa'_{is})^2}, \quad \kappa'_{iq} = \frac{\exp(-\|z_i - \phi_q\|^2)}{\sum_s \exp(-\|z_i - \phi_s\|^2)} \quad (53)$$

5. Perte contrastive de voisinage (Neighborhood Contrastive Loss) :

$$\mathcal{L}_{\text{neighbor}} = -\frac{1}{n} \sum_{i=1}^n \log \frac{\sum_{j=1}^k \exp(\delta(\tilde{z}_i, \nu_{i,j})/\tau)}{\sum_{s=1}^n \exp(\delta(\tilde{z}_i, z_s)/\tau)} \quad (54)$$

où $\{\nu_{i,1}, \dots, \nu_{i,k}\}$ sont les k plus proches voisins de z_i dans une memory bank globale, $\tilde{z}_i$ est la représentation de la version corrompue, et $\delta(\cdot, \cdot)$ est la similarité cosinus.

Notations du bloc scNAME :

- n : nombre de cellules ; p : nombre de gènes ; K : nombre de clusters ;
- X^c : matrice d'entrée (comptes, potentiellement corrompus selon l'étape) ;
- $(\hat{\pi}_{ij}, \hat{\mu}_{ij}, \hat{\theta}_{ij})$: paramètres ZINB prédits pour la cellule i , gène j ;
- $M_{ij} \sim \text{Bernoulli}(p_{\text{mask}})$: masque binaire de corruption initial ;
- $M_{ij}^* = \mathbb{I}[\tilde{X}_{ij} \neq X_{ij}]$: masque effectif utilisé dans $\mathcal{L}_{\text{mask}}$;
- p_{mask} : taux de corruption ;
- α, β, γ : poids des pertes auxiliaires $\mathcal{L}_{\text{mask}}, \mathcal{L}_{\text{neighbor}}, \mathcal{L}_{\text{soft}}$ (dans cet ordre dans le code) ;
- ϕ_q : centroïde latent du cluster q ;
- κ_{iq} : poids d'appartenance souple de la cellule i vers le cluster q , obtenu par “sharpening” de κ'_{iq} ;
- τ : température contrastive (plus τ est faible, plus le contraste est “dur”).

Hyperparamètres sensibles – scNAME

Paramètre	Défaut	Criticité	Description
n_clusters (K)	0	Critique	Nombre de clusters cible. 0 = auto.
temperature (τ)	0.5	Critique	Température du contrastif. $\tau \downarrow$ = plus discriminant.
p_mask	0.5	Important	Proportion de gènes masqués (<i>corrupted</i>).
α, β, γ	1.0	Important	Balance les quatre termes de perte.
n_neighbors (k)	20	Modéré	Nombre de voisins pour la perte contrastive de voisinage.

Mode d'exécution : Inductif.

- *Training* : Apprentissage contrastif sur *memory bank* + reconstruction.
- *Prediction* : $y = \arg \min_j \|E(x) - \mu_j\|^2$. Assignment au centroïde μ_j le plus proche appris durant le *training*.

Note sur les données : scNAME attend des **comptes bruts** et applique une normalisation interne spécifique. L'utilisation de gènes HVG en amont est recommandée.

Adaptations pour le Benchmarking Inductif (Spécifique SCRBenchmark)

Pour permettre une évaluation rigoureuse en mode *Benchmark* (Train/Test) :

1. **Capacité d'Inférence (*Transfer Learning*)** : Ajout d'une méthode `predict()` projetant les nouvelles cellules dans l'espace latent, puis assignment au centroïde le plus proche (distance Euclidienne) :

$$y_{test} = \arg \min_j \|E(x_{test}) - \mu_j\|^2 \quad (55)$$

2. **Normalisation Robuste (Fix « Output Size Mismatch »)** : En mode inférence, certaines cellules du Test set peuvent avoir une expression nulle pour tous les gènes sélectionnés.
 - *Comportement standard (Scanpy)* : `sc.pp.normalize_per_cell` supprime ces cellules.
 - **Solution SCRBenchmark** : Normalisation manuelle gérant les cellules à somme nulle sans les supprimer ($N_{out} = N_{in}$).

4.2.4 Enhanced Autoencoder + Leiden

Principe. L'Enhanced Autoencoder est un algorithme de clustering en deux temps : il apprend d'abord une représentation latente compressée des profils d'expression cellulaire via un auto-encodeur profond, puis applique un algorithme de clustering (Leiden, K-Means ou Overclustering) sur cet espace latent. Sa particularité réside dans un **entraînement en deux phases** avec pondération par fréquence inverse, conçu pour améliorer la représentation des populations cellulaires rares.

Définitions préalables

- **Auto-encodeur (AE)**. Réseau de neurones composé d'un encodeur f_θ (c) d'un décodeur g_ϕ (reconstruction). La représentation latente $z = f_\theta(x)$ capture la biologie essentielle.
- **Espace latent**. Espace de dimension réduite (64 par défaut) où chaque cellule est représentée par un vecteur z_i .
- **MSE**. Erreur quadratique moyenne entre l'entrée x et sa reconstruction $\hat{x}$.
- **Epoch**. Passage complet sur l'ensemble des données d'entraînement.
- **Mini-batch**. Sous-ensemble de cellules (256 par défaut) traité à chaque étape.

Rappels méthodologiques.

Architecture du réseau. Le réseau est un auto-encodeur profond symétrique. L'encodeur compresse progressivement la dimensionnalité de l'entrée (n_{genes}) vers l'espace latent ($z_{\text{dim}} = 64$), tandis que le décodeur effectue l'opération inverse. Chaque couche cachée applique quatre opérations successives : transformation linéaire, normalisation par batch, activation ELU et dropout.

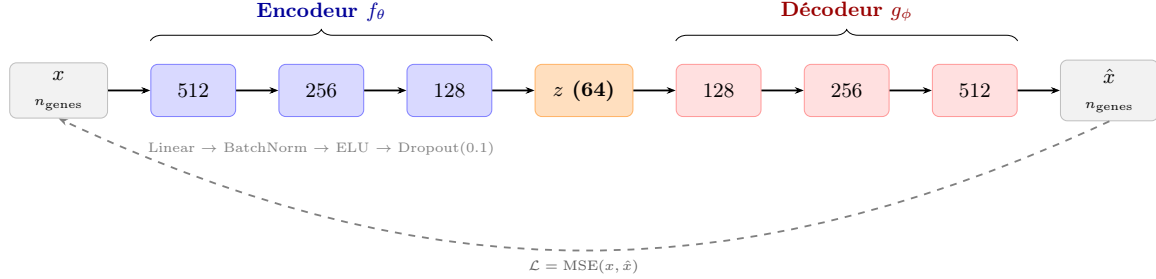


FIGURE 2 – Architecture de l'Enhanced Autoencoder. L'encodeur f_θ compresse le vecteur d'expression de n_{genes} dimensions vers un espace latent de 64 dimensions. Le décodeur g_ϕ reconstruit le vecteur original. La perte de reconstruction (MSE) guide l'optimisation.

L'extrait de code suivant illustre la construction de l'encodeur (méthode `_build_model()`) :

```

1  # Pour chaque dimension h_dim dans [512, 256, 128] :
2  enc_layers.extend([
3      nn.Linear(prev_dim, h_dim),      # Transformation linéaire
4      nn.BatchNorm1d(h_dim),           # Normalisation par batch
5      nn.ELU(),                        # Fonction d'activation
6      nn.Dropout(0.1),                 # Regularisation (dropout)
7  ])
8  # Projection finale vers l'espace latent
9  enc_layers.append(nn.Linear(128, z_dim)) # 128 -> 64

```

Listing 2 – Construction des couches de l'encodeur (extrait)

Pipeline d'entraînement. La méthode `fit()` exécute un entraînement en deux phases suivi d'un clustering post-hoc sur les embeddings latents appris. La figure ?? résume ce processus.

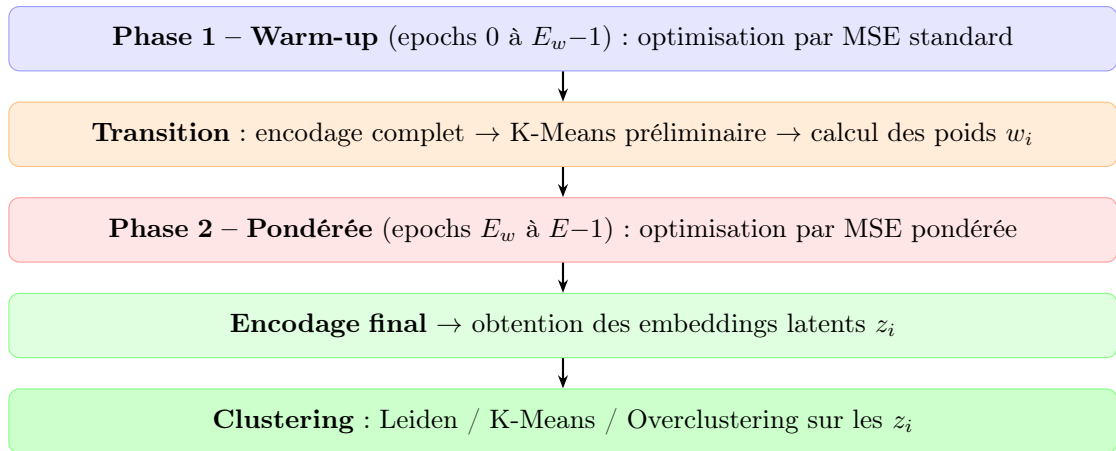


FIGURE 3 – Pipeline d'entraînement de l'Enhanced Autoencoder. $E_w = \text{warmup_epochs}$ (défaut 30), $E = \text{epochs}$ (défaut 100).

Phase 1 : warm-up. Durant les E_w premières epochs (30 par défaut), le réseau est entraîné avec une perte MSE standard :

$$\mathcal{L}_{\text{warmup}} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2 \quad (56)$$

où :

- N est le nombre total de cellules d’entraînement ;
- $x_i \in \mathbb{R}^G$ est le vecteur d’expression de la cellule i (avec G gènes) ;
- $\hat{x}_i$ est la reconstruction produite par le décodeur.

Cette phase initiale est nécessaire afin que l’espace latent atteigne une structuration suffisante **avant** le calcul des poids de pondération. Sans cette étape, les assignations de clusters préliminaires seraient effectuées sur un espace latent non encore structuré et ne refléteraient pas les véritables proportions de populations cellulaires.

L’extrait de code correspondant :

```

1 for epoch in range(warmup_epochs):           # 30 epochs par défaut
2     for batch_x, batch_idx in loader:         # mini-batches de 256
3         cellules
4         optimizer.zero_grad()
5         recon, _ = model(batch_x)             # forward : x -> z -> x_hat
6         loss = mse_loss(recon, batch_x)       # erreur de reconstruction
7         loss.backward()                       # retropropagation des
8         gradients
9         optimizer.step()                     # mise a jour des parametres

```

Listing 3 – Phase 1 : boucle warm-up (MSE standard)

Transition : calcul des poids par fréquence inverse. À l’issue du warm-up, les poids de pondération par cellule sont calculés selon la procédure suivante :

1. Encodage de l’ensemble des cellules d’entraînement pour obtenir les vecteurs latents z_i
2. Application d’un K-Means préliminaire (k clusters) sur les z_i pour estimer les assignations
3. Calcul de la fréquence relative de chaque cluster : $\text{freq}(c) = n_c/N$
4. Attribution d’un poids à chaque cellule selon la formule :

$$w_i = \left(\frac{1}{\text{freq}(c_i)} \right)^\alpha \quad \text{avec } \alpha = \text{weight_exponent} \text{ (défaut : 0.5)} \quad (57)$$

où :

- c_i est le cluster préliminaire assigné à la cellule i (via K-Means sur l’espace latent) ;
- $\text{freq}(c_i) = n_{c_i}/N$, avec n_{c_i} le nombre de cellules du cluster c_i ;
- α contrôle l’intensité de la pondération (plus α est grand, plus les clusters rares sont sur-pondérés).

Deux post-traitements sont appliqués : (i) normalisation des poids pour que $\bar{w} = 1$, et (ii) plafonnement à `max_cell_weight` (défaut : 10.0) pour prévenir les instabilités numériques.

Illustration numérique du calcul des poids. Le tableau ?? illustre le mécanisme de pondération sur un exemple avec trois clusters de proportions inégales et $\alpha = 0.5$.

TABLE 1 – Exemple de calcul des poids pour 1000 cellules réparties en 3 clusters ($\alpha = 0.5$).

	Cluster A	Cluster B	Cluster C (rare)
Nombre de cellules	600	300	100
Fréquence f	0.60	0.30	0.10
$(1/f)^{0.5}$ (brut)	1.29	1.83	3.16
Poids normalisé w	0.64	0.91	1.58

Les cellules du cluster rare (C) contribuent ainsi $1.58/0.64 \approx 2.5\times$ plus à la fonction de perte que celles du cluster majoritaire (A), améliorant la qualité de leur représentation latente.

Phase 2 : entraînement pondéré. Durant les epochs E_w à $E-1$, la perte MSE est pondérée par cellule :

$$\mathcal{L}_{\text{weighted}} = \frac{1}{B} \sum_{i \in \text{batch}} w_i \cdot \underbrace{\frac{1}{G} \sum_{g=1}^G (\hat{x}_{ig} - x_{ig})^2}_{\ell_i \text{ (MSE par cellule)}} \quad (58)$$

où :

- B est la taille du mini-batch ;
- G est le nombre de gènes (features d’entrée) ;
- x_{ig} et $\hat{x}_{ig}$ sont respectivement la valeur vraie et reconstruite du gène g pour la cellule i ;
- ℓ_i est la MSE de la cellule i ;
- w_i est le poids de la cellule i , calculé à la transition warm-up $\rightarrow$ phase pondérée.

L’implémentation correspondante :

```

1 for epoch in range(warmup_epochs, epochs): # epochs 30 a 99
2     for batch_x, batch_idx in loader:
3         batch_weights = weights_tensor[batch_idx] # poids de chaque
           cellule
4         optimizer.zero_grad()
5         recon, _ = model(batch_x)
6         per_sample_loss = ((recon - batch_x) ** 2).mean(dim=1) # MSE
           / cellule
7         loss = (per_sample_loss * batch_weights).mean() #
           ponderation
8         loss.backward()
9         optimizer.step()

```

Listing 4 – Phase 2 : MSE avec pondération par cellule

Note d’implémentation. Le `DataLoader` retourne des paires (`batch_x`, `batch_idx`) car le mélange aléatoire des mini-batches (`shuffle=True`) nécessite de pouvoir associer chaque cellule à son poids w_i via son indice original : `batch_weights = weights_tensor[batch_idx]`.

Détermination du nombre de clusters. Le paramètre `n_clusters` conditionne le comportement de l’algorithme selon la logique suivante :

- `n_clusters > 0` : la valeur spécifiée est utilisée directement.
- `n_clusters = 0` avec labels disponibles : k est déduit du nombre de classes dans les labels de vérité terrain (**mode Oracle**).
- `n_clusters = 0` sans labels : valeur par défaut $k = 8$.

Clustering sur l'espace latent. Après l'entraînement, le modèle encode l'ensemble des cellules et applique l'une des trois méthodes de clustering suivantes sur les embeddings latents :

1. Leiden ('leiden', par défaut). Détection de communautés sur un graphe de k -plus-proches voisins (KNN) construit dans l'espace latent avec `leiden_neighbors` = 15 voisins. La résolution est déterminée selon l'algorithme ??.

Algorithm 1 Sélection de la résolution Leiden

```

1: if leiden_resolution > 0 then
2:   Utiliser directement la résolution spécifiée
3: else if  $k > 0$  (nombre cible de clusters connu) then
4:   Recherche par balayage de  $r = 2.5$  à  $0.01$  (pas de  $0.02$ , ordre décroissant)
5:   Retenir la résolution produisant le nombre de clusters le plus proche de  $k$ 
6:   Arrêt anticipé si correspondance exacte
7: else
8:   Résolution par défaut  $r = 1.0$ 
9: end if

```

2. K-Means ('kmeans'). Partitionnement classique autour de k centroïdes dans l'espace latent. Cette méthode suppose implicitement des clusters de géométrie convexe.

3. Overclustering + Merge ('overclustering'). Stratégie en deux étapes :

1. K-Means initial avec $k_{\text{over}} = \max(k+1, \lfloor k \times \text{overcluster_factor} \rfloor)$ clusters
2. Fusion itérative par similarité cosinus entre centroïdes, jusqu'à atteindre k clusters

où k est le nombre de clusters cible et `overcluster_factor` (défaut 2.0) fixe le niveau de sur-segmentation initiale. Cette approche permet de capturer des clusters de géométrie non convexe en découpant finement puis en fusionnant les fragments proches.

Prédiction sur données non vues. L'inférence sur de nouvelles cellules s'effectue par transfert de labels via KNN dans l'espace latent :

$$\hat{y}_{\text{new}} = \text{KNN}_{k=\min(15, N_{\text{train}}-1)}(f_{\theta}(x_{\text{new}}), Z_{\text{train}}, y_{\text{train}}^{\text{cluster}}) \quad (59)$$

où :

- $\hat{y}_{\text{new}}$ est le label de cluster prédit pour une cellule non vue ;
- x_{new} est la cellule d'entrée (espace gène) ;
- f_{θ} est l'encodeur appris ;
- $f_{\theta}(x_{\text{new}})$ est l'embedding latent de la nouvelle cellule ;
- Z_{train} est la matrice des embeddings latents du train ;
- $y_{\text{train}}^{\text{cluster}}$ sont les labels de clusters appris sur le train ;
- N_{train} est le nombre de cellules de train (et donc $k = \min(15, N_{\text{train}} - 1)$).

Ce mécanisme est appliqué indépendamment de la méthode de clustering utilisée à l'entraînement. Le paramètre `use_raw_data` n'affecte que la phase `fit()` ; dans `predict()`, les données sont lues depuis `data.X`.

Distinction avec les approches à perte jointe

Contrairement aux méthodes telles que scDeepCluster ou DEC, qui incluent le clustering directement dans la fonction de perte du réseau (optimisation $\text{loss} + \text{clustering}$), l'Enhanced Autoencoder optimise **exclusivement** la reconstruction. Le clustering est appliqué **a posteriori** sur les embeddings latents. L'hypothèse est que la pondération par fréquence inverse produit un espace latent suffisant pour que les populations rares soient correctement représentées, sans nécessiter de clustering explicite, avec l'objectif de clustering.

Positionnement méthodologique.

Comparaison des fonctions de perte : MSE vs ZINB. L'Enhanced Autoencoder utilise la MSE comme fonction de perte, tandis que scDeepCluster repose sur la distribution ZINB (cf. section 4.2.1). Ce choix a des implications directes sur le type de données attendu en entrée et sur la nature des représentations apprises. Le tableau ?? résume les différences.

TABLE 2 – Comparaison des stratégies de reconstruction : MSE (Enhanced AE) vs ZINB (scDeepCluster).

Critère	MSE (Enhanced AE)	ZINB (scDeepCluster)
Données d'entrée	Données prétraitées (normalisées, log-transformées, mises à l'échelle)	Comptes bruts (<i>raw counts</i>), avec <i>size factor</i> par cellule
Hypothèse statistique	Bruit gaussien implicite (erreur quadratique)	Modèle de comptage : surdispersion + excès de zéros
Gestion des zéros	Pas de distinction : un zéro est traité comme toute autre valeur	Distinction explicite via le paramètre π (dropout technique vs absence réelle)
Sorties du décodeur	Un seul vecteur $\hat{x} \in \mathbb{R}^G$	Trois vecteurs : $\mu, \theta, \pi \in \mathbb{R}^G$
Complexité du modèle	Plus simple (une seule tête de sortie)	Plus riche (trois têtes spécialisées + <i>size factor</i>)
Avantage principal	Rapidité, simplicité, compatibilité avec les pipelines de prétraitement standard (scanpy)	Modélisation fidèle du processus génératif des données scRNA-seq
Stratégie pour les cellules rares	Pondération par fréquence inverse (w_i) en phase 2	Pas de mécanisme spécifique (les cellules rares reçoivent le même poids)
Perte de clustering	Aucune (clustering post-hoc)	KL divergence intégrée à l'entraînement (DEC)

En résumé, la ZINB est théoriquement plus adaptée aux données scRNA-seq car elle modélise explicitement les spécificités statistiques des comptes de lecture (surdispersion, dropouts). L'Enhanced AE fait le choix inverse : il s'appuie sur un prétraitement standard (normalisation + log + scaling) pour rapprocher les données d'une distribution continue, puis compense l'absence de modélisation statistique fine par une **pondération explicite des cellules rares**. Ces deux stratégies représentent des compromis différents entre complexité du modèle et simplicité du pipeline.

Comportements limites.

- Si `warmup_epochs` $\geq$ `epochs` : la phase pondérée n'est pas exécutée (entraînement MSE standard uniquement).
- Aucun mécanisme d'*early stopping* n'est implémenté : l'entraînement s'exécute pour le nombre exact d'epochs spécifié.

- La reproductibilité est assurée par initialisation déterministe des générateurs (`torch.manual_seed`, `np.random.seed`).

État interne après entraînement.

Attribut	Type	Description
<code>self.model</code>	EnhancedAE	Réseau auto-encodeur avec l'ensemble des paramètres appris
<code>self._embeddings</code>	<code>np.ndarray</code>	Embeddings latents z_i des cellules d'entraînement
<code>self._labels</code>	<code>np.ndarray</code>	Labels de clusters attribués aux cellules d'entraînement
<code>self._train_embeddings</code>	<code>np.ndarray</code>	Copie des embeddings servant de référence pour le KNN

Données d'entrée.

- Si `use_raw_data=True` et `data.layers['original_X']` est disponible : entraînement sur les données brutes.
- Sinon : entraînement sur `data.X` (données prétraitées : normalisées, log-transformées, mises à l'échelle).

Hyperparamètres sensibles – Enhanced AE

Paramètre	Défaut	Criticité	Description
<code>weight_exponent</code> (α)	0.5	Critique	Exposant de la pondération par fréquence inverse. 0 = pas de pondération, 0.5 = modérée, 1.0 = forte.
<code>n_clusters</code>	0	Critique	Nombre de clusters cible. 0 = auto (Oracle ou $k=8$).
<code>leiden_resolution</code>	0.0	Critique	Résolution Leiden. 0.0 déclenche la recherche automatique.
<code>leiden_neighbors</code>	15	Critique	Nombre de voisins pour le graphe KNN (Leiden).
<code>warmup_epochs</code>	30	Important	Nombre d'epochs de warm-up avant pondération.
<code>epochs</code>	100	Important	Nombre total d'epochs d'entraînement.
<code>max_cell_weight</code>	10.0	Important	Plafond des poids pour la stabilité numérique.
<code>clustering_method</code>	leiden	Modéré	Méthode : <code>leiden</code> , <code>kmeans</code> , <code>overclustering</code> .
<code>overcluster_factor</code>	2.0	Modéré	Facteur multiplicatif en mode overclustering.
<code>use_raw_data</code>	False	Modéré	Utiliser <code>layers['original_X']</code> si disponible.

4.2.5 scProto

Référence : Hashemi Golpayegani et al. [?], « scProto : Interpretable Self-Supervised Prototype Learning for Single-Cell Transcriptomics », LMRL Workshop, ICLR, 2025.

Principe : scProto est une méthode d'apprentissage auto-supervisé combinant un auto-encodeur variationnel conditionnel (scPoli) avec un objectif de clustering contrastif basé sur des prototypes (SwAV). L'algorithme apprend simultanément une représentation latente robuste aux

effets de batch et un ensemble de prototypes représentant les états cellulaires, tout en maximisant la cohérence entre différentes vues augmentées d’une même cellule.

Fonction de perte totale :

$$\mathcal{L} = \mathcal{L}_{\text{CVAE}} + \mathcal{L}_{\text{SwAV}} + \lambda_{\text{prop}} \mathcal{L}_{\text{prop}} + \lambda_{\text{sim}} \mathcal{L}_{\text{sim}} \quad (60)$$

1. Perte CVAE (scPoli) : Assure la reconstruction des données et la correction de batch via l’encodeur scPoli.

$$\mathcal{L}_{\text{CVAE}} = \mathcal{L}_{\text{recon}} + \beta \mathcal{L}_{\text{KL}} + \mathcal{L}_{\text{MMD}} \quad (61)$$

2. Perte SwAV (Clustering Contrastif) : Pousse les représentations latentes des différentes vues augmentées d’une cellule (par exemple, obtenues par masquage ou marche aléatoire sur graphe KNN) à être assignées aux mêmes prototypes. L’assignation optimale Q est calculée via l’algorithme de Sinkhorn-Knopp.

$$\mathcal{L}_{\text{SwAV}} = - \sum_i \sum_k q_{ik} \log p_{ik} \quad (62)$$

3. Régularisations :

- $\mathcal{L}_{\text{prop}}$ (Propagation) : Force les prototypes à couvrir l’ensemble de l’espace latent (y compris les populations rares) en maximisant la distance minimale entre cellules et prototypes.
- $\mathcal{L}_{\text{sim}}$ (Embedding Similarity) : Encourage la diversité des prototypes pour éviter l’effondrement trivial.

Hyperparamètres sensibles – scProto

Paramètre	Défaut	Criticité	Description
num_prototypes	300	Critique	Nombre de prototypes apprenables.
temperature	0.05	Critique	Température softmax pour l’assignation SwAV.
epsilon	0.02	Important	Régularisation Sinkhorn-Knopp.
n_augmentations	8	Important	Nombre de vues augmentées par cellule.
augmentation_type	knn	Modéré	Type d’augmentation : knn ou mask .
latent_dims	8	Modéré	Dimension de l’espace latent scPoli (par défaut faible pour SwAV).
clustering_method	leiden	Critique	Clustering final dans l’espace latent : leiden (défaut) ou kmeans .
n_clusters	0	Critique	Nombre cible de clusters. En mode auto avec labels disponibles, la résolution est optimisée par ARI.
leiden_resolution	0.0	Critique	Résolution manuelle (si > 0) ou automatique (si = 0).
leiden_neighbors	15	Important	Nombre de voisins pour le graphe KNN utilisé par Leiden.
leiden_resolution_min/max/step	0.1/2.5/0.1	Modéré	Grille de recherche de résolution Leiden en mode auto-optimal.
leiden_silhouette_max_cells	5000	Modéré	Taille max de sous-échantillon pour le score silhouette pendant la recherche auto.

Mode d’exécution : Transductif (benchmark) avec **support de transfert pratique** hors benchmark.

Clustering final (implémentation actuelle). Après entraînement SwAV/scPoli, les embeddings latents normalisés sont clusterisés comme suit :

- **Leiden** (défaut) :
 - si `leiden_resolution > 0` : utilisation directe de la valeur manuelle ;
 - sinon, si les labels ground-truth sont fournis : **optimisation automatique** de la résolution en maximisant l'ARI sur la grille `[leiden_resolution_min, leiden_resolution_max]` avec pas `leiden_resolution_step` ;
 - sinon, si `n_clusters > 0` : recherche d'une résolution produisant un nombre de clusters proche de la cible ;
 - sinon (`n_clusters = 0` et labels absents) : optimisation automatique par *silhouette score* ; pour les grands jeux, le score est calculé sur un sous-échantillon borné par `leiden_silhouette_max_cells`.
- **K-Means** : utilisé si `clustering_method='kmeans'` ou en fallback si Leiden n'est pas disponible.

Prédiction sur nouvelles données. La méthode `predict()` n'exécute pas un re-clustering complet à chaque appel.

- Si les cellules fournies correspondent au dataset d'entraînement (mêmes `obs_names`) : retour direct des labels appris.
- Sinon : projection dans l'espace latent via l'encodeur scPoli.
- Si l'entraînement a été fait avec K-Means : assignation via `KMeans.predict()`.
- Si l'entraînement a été fait avec Leiden : transfert des labels par KNN sur (Z_{train}, y_{train}) .

Remarque méthodologique. Même si un transfert de labels est implémenté pour l'usage pratique, scProto reste déclaré `supports_out_of_sample=False` dans le benchmark afin de conserver un protocole transductif cohérent avec la nature graphe de Leiden.

4.2.6 scHNTL (High-order Neighbors and Triplet Loss)

Référence : Meng, Qin & Long [?], « scHNTL : single-cell RNA-seq data clustering augmented by high-order neighbors and triplet loss », Bioinformatics, 2025.

Principe : Améliorer la séparabilité des embeddings cellulaires en exploitant les voisins d'ordre supérieur pour identifier les similarités subtiles entre cellules, et en utilisant une triplet loss pour éloigner les cellules dissimilaires tout en rapprochant les cellules similaires.

Fonction de perte totale :

$$\mathcal{L} = c_1 \mathcal{L}_r + c_2 \mathcal{L}_c + c_3 \mathcal{L}_{\text{triplet}} \quad (63)$$

1. Construction du graphe avec débruitage :

La similarité initiale est calculée par coefficient de Pearson, puis débruitée par Network Enhancement (NE). La matrice d'adjacence $A_{n \times n}$ est obtenue par K -plus-proches-voisins sur la matrice débruitée $E_{n \times n}$.

2. Voisins d'ordre supérieur (implémentation effective) :

Le code construit explicitement une matrice de voisins d'ordre 2 :

$$A^{(2)} = \mathbf{1}[A^2 > 0], \quad B = \max(A^{(2)} - I - A, 0), \quad A' = A + 0.5(B - B \odot A) \quad (64)$$

où A est l'adjacence KNN binaire, I l'identité, et $\odot$ le produit élément par élément.

3. Graph Attentional Autoencoder :

La couche d'attention avec noyau gaussien :

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j \right) \quad (65)$$

Coefficients d'attention (forme exacte du script) :

$$s_i = a_s^T W h_i, \quad t_j = a_n^T W h_j, \quad \text{dense}_{ij} = \exp(-(s_i - t_j)^2), \quad m_{ij} = 0.5 (A_{ij} + 1) |A_{ij}| \quad (66)$$

$$\alpha_{ij} = \frac{\exp(\text{dense}_{ij}) m_{ij}}{\sum_k \exp(\text{dense}_{ik}) m_{ik}} \quad (67)$$

Mécanisme multi-tête :

$$h'_i = \bigoplus_{t=1}^T \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^t W^t h_j \right) \quad (68)$$

Perte de reconstruction (MAE) :

$$\mathcal{L}_r = \frac{1}{nD} \sum_{i=1}^n \sum_{j=1}^D |Y_{ij} - Y'_{ij}| \quad (69)$$

4. Triplet Loss basée sur les voisins d'ordre supérieur :

$$\mathcal{L}_{\text{triplet}} = \frac{1}{n} \sum_{i=1}^n \max(-\bar{s}_i^+ + \bar{s}_i^- + 0.2, 0) \quad (70)$$

avec $\bar{s}_i^+ = \frac{1}{n} \sum_j A'_{ij} S_{ij}$, $\bar{s}_i^- = \frac{1}{n} \sum_j B_{ij} S_{ij}$, S la matrice de corrélation des embeddings latents, et $B = \mathbf{1}[A' = 0]$.

5. Self-optimizing Clustering (DEC) :

Distribution d'appartenance (Student-t) :

$$q_{ij} = \frac{(1 + \|z_i - u_j\|^2 / \alpha)^{-1}}{\sum_{k=1}^c (1 + \|z_i - u_k\|^2 / \alpha)^{-1}} \quad (71)$$

Distribution cible :

$$p_{ij} = \frac{q_{ij}^2 / f_j}{\sum_{j'=1}^c q_{ij'}^2 / f_{j'}} \quad \text{avec} \quad f_j = \sum_{i=1}^n q_{ij} \quad (72)$$

Perte de clustering (KL divergence) :

$$\mathcal{L}_c = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (73)$$

Notations du bloc scHNTL :

- n : nombre de cellules ; D : nombre de gènes ; c : nombre de clusters ;
- A : matrice d'adjacence KNN initiale ; $A^{(2)}$: connectivité d'ordre 2 ; A' : matrice de voisins enrichie ;
- B : matrice des paires négatives ($B = \mathbf{1}[A' = 0]$) ;
- h_i : représentation de la cellule i ; W : matrice de projection ; $\mathcal{N}(i)$: voisinage de i ;

- a_s, a_n : vecteurs d’attention “self” et “neighbor” appris ; $dense_{ij}$: score gaussien intermédiaire ; m_{ij} : masque d’adjacence ; α_{ij} : coefficient d’attention normalisé ; T : nombre de têtes d’attention ;
- Y, Y' : matrices d’entrée et reconstruite (perte MAE) ;
- S : matrice de corrélation des embeddings ; $\bar{s}_i^+, \bar{s}_i^-$: moyennes positive/négative par cellule ; marge fixée à 0.2 dans le code ;
- c_1, c_2, c_3 : poids des pertes reconstruction / clustering / triplet ;
- q_{ij} : attribution souple (Student-t) de la cellule i au centroïde u_j ;
- p_{ij} : distribution cible “sharpened” ; α : paramètre de la Student-t (défaut 1).

Hyperparamètres sensibles – scHNTL

Paramètre	Défaut	Criticité	Description
n_clusters (c)	0	Critique	Nombre de clusters cible.
K	auto	Important	Voisins pour le graphe. $K = \text{round}(N/(10 \times c))$, borné [6, 20].
n_attn_heads (T)	4	Modéré	Nombre de têtes d’attention.
margin	0.2	Modéré	Marge de la <i>triplet loss</i> .
c_1, c_2, c_3	1.0	Modéré	Poids des pertes reconstruction / clustering / triplet.
learning_rate	0.0002	Modéré	LR pré-entraînement (0.0005 en entraînement joint).
PCA_dim, F1, F2	512, 64, 16	Modéré	Architecture du réseau (dimensions).

Mode d’exécution : Transductif.

- *Mécanisme* : La construction du graphe (K -NN + Network Enhancement) et le calcul des voisins d’ordre supérieur (A') se font sur la matrice X entière.
- *Benchmark* : L’évaluation sur le Test Set nécessite un `fit()` complet sur X_{test} (pas de prédiction *out-of-sample*).

Note sur les données : scHNTL accepte des données brutes (normalisation interne CPP100K + Log2) ou prétraitées. PCA interne (512 dims) ; sélection HVG en amont conseillée.

4.2.7 scMUG (Multiple Gene Functional Modules)

Référence : Liang & Du [?], « scMUG : deep clustering analysis of single-cell RNA-seq data on multiple gene functional modules », Briefings in Bioinformatics, 2025.

Principe : Première méthode à intégrer les modules fonctionnels de gènes (GFMs) dans le clustering scRNA-seq. Utilise un autoencodeur basé sur la distribution ZINB par GFM, combiné avec une nouvelle mesure de similarité cellulaire intégrant densité locale et distribution globale.

Pipeline :

1. Prétraitement et construction du réseau de co-expression génique
2. Chargement des GFMs depuis des fichiers (external/original_code) ou génération par corrélation/expansion si aucun fichier n’est fourni
3. Génération des représentations cellulaires par autoencodeur ZINB pour chaque GFM
4. Construction des matrices de similarité (globale et locale)
5. Clustering spectral sur la similarité combinée

1. Prétraitement des données :

Normalisation :

$$y_{i,j} = \ln \left(10^5 \cdot \frac{x_{i,j}}{\sum_{j=1}^{d_z} x_{i,j}} + 1 \right) \quad (74)$$

Z-transformation :

$$z_{i,j} = \frac{y_{i,j} - \mu_j}{\sigma_j} \quad (75)$$

2. Construction du réseau de co-expression :

Transformation de Fisher sur les corrélations de Pearson :

$$\rho_{u,v} = \frac{1}{2} \ln \left(\frac{1 + r_{u,v}}{1 - r_{u,v}} \right) \quad (76)$$

Le sous-graphe core est extrait avec un seuil c tel que $|\rho_{u,v}| \geq c$. Dans l'implémentation SCRBenchmark, on conserve la **plus grande composante connexe** au-dessus de ce seuil (pas de contrainte explicite n_d).

3. Autoencodeur ZINB (par GFM) :

Architecture : 7 couches fully-connected (n_g , 512, 128, 32, 128, 512, n_g).

Distribution ZINB :

$$\text{ZINB}(x|\pi, \mu, \theta) = \pi \cdot \delta_0(x) + (1 - \pi) \cdot \text{NB}(x|\mu, \theta) \quad (77)$$

Distribution Binomiale Négative :

$$\text{NB}(x|\mu, \theta) = \frac{\Gamma(x + \theta)}{\Gamma(x + 1)\Gamma(\theta)} \left(\frac{\theta}{\theta + \mu} \right)^\theta \left(\frac{\mu}{\theta + \mu} \right)^x \quad (78)$$

Perte ZINB :

$$\mathcal{L}_{\text{ZINB}} = - \sum_{i=1}^{n_g} \ln \text{ZINB}(x_i|\pi_i, \mu_i, \theta_i) \quad (79)$$

où les paramètres μ_i , θ_i , π_i sont estimés par le décodeur.

4. Similarité de distribution globale :

Score global pour la cellule i dans le clustering t du GFM c :

$$s_{t,c}(i) = \sqrt{\frac{|d_{0,t,c}(i) - d_{1,t,c}(i)|}{d_{0,t,c}(i) + d_{1,t,c}(i)}} \quad (80)$$

où d_0 est la distance au centre du cluster d'appartenance, et d_1 au centre du cluster le plus proche différent.

Similarité globale normalisée :

$$s_0(i, j) = \frac{1}{n_k \cdot n_c} \sum_{c=1}^{n_c} \sum_{t=1}^{n_k} s_{t,c}(i) \cdot s_{t,c}(j) \cdot \mathbb{I}_{l(i,t,c)=l(j,t,c)} \quad (81)$$

$$s_g(i, j) = \frac{s_0(i, j)}{m_g}, \quad m_g = \frac{1}{n^2} \sum_{i,j} s_0(i, j) \quad (82)$$

5. Similarité de densité locale :

Distance moyenne aux k-plus-proches-voisins :

$$d_{c,i} = \frac{1}{k} \sum_{t \in N_k(c,i)} d_{c,i,t} \quad (83)$$

Score de densité locale :

$$s_d(c, i, j) = \sqrt{\min \left(\frac{d_{c,i,j}}{d_{c,i}}, \frac{d_{c,i}}{d_{c,i,j}} \right) \cdot \min \left(\frac{d_{c,i,j}}{d_{c,j}}, \frac{d_{c,j}}{d_{c,i,j}} \right)} \quad (84)$$

Mesure de dispersion locale :

$$\varphi(c, i) = \sqrt{\ln(\text{var}_{t \in N_k(i)}(d_{c,i,t}) + e)} \quad (85)$$

Similarité locale finale :

$$s_d(i, j) = \max_c \left(\frac{s_d(c, i, j)}{\varphi(c, i) \cdot \varphi(c, j)} \right) \quad (86)$$

6. Combinaison et clustering spectral :

$$\boxed{s(i, j) = \alpha \cdot s_g(i, j) + \beta \cdot s_d(i, j)} \quad (87)$$

Notations du bloc scMUG :

- $x_{i,j}$: compte brut du gène j dans la cellule i ; d_z : nombre de gènes considérés ;
- $y_{i,j}$: expression normalisée/log-transformée ; $z_{i,j}$: version z-score (μ_j, σ_j : moyenne/écart-type du gène j) ;
- $r_{u,v}$: corrélation de Pearson entre gènes u, v ; $\rho_{u,v}$: corrélation stabilisée (Fisher) ;
- π, μ, θ : paramètres ZINB (inflation de zéros, moyenne, dispersion) ;
- n_g : nombre de gènes dans un module fonctionnel (GFM) ;
- $d_{0,t,c}(i)$: distance de i au centroïde de son cluster ; $d_{1,t,c}(i)$: distance au centroïde rival le plus proche ;
- n_k : nombre de répétitions K-means ; n_c : nombre de GFMs ; $l(i, t, c)$: label de cluster de i au run t , module c ;
- $\mathbb{I}$: indicatrice (1 si condition vraie, 0 sinon) ;
- $d_{c,i,j}$: distance entre cellules i, j dans l'espace du module c ; $N_k(c, i)$: k -voisins de i pour le module c ;
- $\varphi(c, i)$: facteur de dispersion locale ; m_g : facteur de normalisation global ;
- $s_g(i, j)$: similarité globale ; $s_d(i, j)$: similarité locale ; $s(i, j)$: similarité finale pour le clustering spectral.

Clustering spectral appliqué sur $S = \{s(i, j)\}_{n \times n}$.

Hyperparamètres sensibles – scMUG

Paramètre	Défaut	Criticité	Description
n_clusters	0	Critique	Nombre de clusters. 0 = auto (mode oracle si labels fournis).
α, β	–	Critique	Poids similarité globale/locale. Option <code>use_alpha_beta_grid</code> pour recherche sur grille.
n_gfms / gfm_threshold	auto	Important	Paramètres de construction des GFMs (nombre et seuils de corrélation).
hidden_dims	512,128,32	Important	Architecture des auto-encodeurs ZINB.
kmeans_times	20	Important	Répétitions K-Means pour la similarité globale.
n_neighbour	3	Modéré	Voisins pour la densité locale.
learning_rate	10^{-4}	Modéré	Taux d'apprentissage.
epochs	50	Modéré	Nombre d'epochs d'entraînement.
bruit gaussien (σ)	0.15	Modéré	Appliqué à chaque couche de l'encodeur.

Mode d'exécution : Transductif.

- *Mécanisme* : Combine similarité globale (via ZINB) et similarité de densité locale dans $S_{combined}$. Clustering spectral final.
- *Benchmark* : Le clustering spectral ne généralise pas (pas de projection). Ré-entraînement nécessaire sur le Test Set.

Note sur les données : scMUG fonctionne sur les **comptes bruts**. Sélection HVG préalable (< 10 000 gènes) fortement recommandée.

4.2.8 scCDCG (Deep Cut-informed Graph Clustering)

Référence : Xu et al. [?], « scCDCG : Efficient Deep Structural Clustering for single-cell RNA-seq via Deep Cut-informed Graph Embedding », arXiv :2404.06167, 2024.

Principe : Exploiter l'information structurelle intercellulaire de haut ordre via un module d'embedding de graphe basé sur les techniques de coupe normalisée (normalized cut), combiné avec un apprentissage auto-supervisé par transport optimal, tout en évitant les problèmes de sur-lissage (over-smoothing) des GNN classiques.

Fonction de perte totale :

$$\mathcal{L}_{\text{pre}} = \lambda_{\text{rec}} \mathcal{L}_x + \lambda_{\text{ort}} \mathcal{L}_{\text{ort}} + \lambda_{\text{cov}} \mathcal{L}_{\text{cov}} \quad (88)$$

et, en phase 2 :

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{pre}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}} \quad (89)$$

avec $\lambda_{\text{rec}} = \text{factor_construct}$, $\lambda_{\text{ort}} = \text{factor_ort}$, $\lambda_{\text{cov}} = \text{factor_corvar}$, $\lambda_{\text{KL}} = \text{factor_KL}$.

1. Termes reconstruction / orthogonalité / covariance :

$$\mathcal{L}_x = \text{MSE}(X, \hat{X}), \quad \mathcal{L}_{\text{ort}} = \text{MSE}\left(\text{vec}(Z^\top Z), \text{vec}(I_d)\right) \quad (90)$$

2. Graphe normalisé (forme implémentée) :

Deux graphes sont construits à partir des données :

- **Graphe des métriques de probabilité** : $C = XX^\top$ (covariance entre cellules)
- **Graphe des métriques spatiales** : $S_{ij} = \frac{x_i^\top x_j}{|x_i|_2 |x_j|_2}$ (similarité cosinus)

Le script utilise la normalisation symétrique suivante :

$$L_1 = D_C^{-1/2} C D_C^{-1/2}, \quad L_2 = D_S^{-1/2} S D_S^{-1/2} \quad (91)$$

Le terme de covariance est :

$$\mathcal{L}_{\text{cov}} = -\frac{1}{d} \text{Tr} \left(Z^\top (\beta L_1 + (1 - \beta) L_2) Z \right) \quad (92)$$

où $\beta = \text{balancer} \in [0, 1]$ et d est la dimension latente.

3. Soft Assignment par distribution Student-t :

$$q_{ij} = \frac{(1 + \|z_i - c_j\|^2 / \alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'} (1 + \|z_i - c_{j'}\|^2 / \alpha)^{-\frac{\alpha+1}{2}}} \quad (93)$$

avec $\alpha = 1$ fixé dans l'implémentation.

4. Perte de clustering par Transport Optimal :

La distribution cible est obtenue par Sinkhorn sur Q^λ :

$$\hat{P} = \text{Sinkhorn} \left(Q^\lambda, r, c \right), \quad r = \mathbf{1}_N, \quad c_j = \#\{i : \tilde{y}_i = j\} \quad (94)$$

Puis la perte de clustering :

$$\mathcal{L}_{\text{KL}} = \text{KLDivLoss}(Q, \hat{P}) \quad (\text{reduction} = \text{'sum' dans le code}) \quad (95)$$

Détail de fidélité au script : `KLDivLoss` est appelé directement sur Q (sans `log_softmax` explicite).

Note d'implémentation. Le script calcule aussi un terme $\text{MSE}(\hat{A}, A)$ (`loss_adj_graph`) mais ne l'ajoute pas à la loss totale dans la version actuelle.

Notations du bloc `scCDCG` :

- $X \in \mathbb{R}^{N \times G}$: matrice d'expression (N cellules, G gènes); $\hat{X}$: reconstruction;
- C, S : matrices d'affinité (probabilité/covariance et similarité cosinus);
- K : nombre de clusters cibles;
- D_C, D_S : matrices diagonales de degrés de C et S ;
- L_1, L_2 : matrices normalisées $D^{-1/2} A D^{-1/2}$ (pas $I - \cdot$ dans ce script);
- $Z \in \mathbb{R}^{N \times d}$: embeddings latents normalisés (d dimension latente);
- $\beta = \text{balancer}$: balance entre les deux termes de graphe;
- $\lambda_{\text{rec}}, \lambda_{\text{ort}}, \lambda_{\text{cov}}, \lambda_{\text{KL}}$: poids des pertes (`factor_construct`, `factor_ort`, `factor_corvar`, `factor_KL`);
- q_{ij} : attribution souple de la cellule i au cluster j , $Q = [q_{ij}]$, avec $\alpha = 1$ fixé pour le noyau Student-t;
- $\hat{P}$: plan de transport cible obtenu par Sinkhorn;
- $\lambda = \text{lambdas}$: exposant appliqué à Q^λ dans Sinkhorn;
- $\tilde{y}$: pseudo-labels K-Means utilisés pour fixer la marge colonne c_j de Sinkhorn.

Hyperparamètres sensibles – scCDCG

Paramètre	Défaut	Criticité	Description
n_clusters	0	Critique	Nombre de clusters. 0 = auto (mode oracle si labels fournis).
balancer	0.5	Critique	Balance entre les deux graphes normalisés.
factor_*	–	Critique	Pondération des pertes (construct, ort, corvar, KL).
lambdas	–	Important	Exposant dans Sinkhorn (Q^λ).
theta	–	Modéré	Exposé dans l'UI mais non effectif (alpha=1 fixé).

Mode d'exécution : Inductif.

- *Training* : Apprentissage conjoint représentation + clustering sur graphe (coupe normalisée + transport optimal).
- *Prediction* : Projection via l'encodeur découplé, puis assignation aux clusters existants.

Note sur les données : scCDCG attend les **comptes bruts** des gènes HVG. Pas de sélection de gènes interne mais normalisation L2.

Adaptations pour le Benchmarking Inductif (Spécifique SCRBenchmark)

Stratégie d'inférence inductive pour rendre scCDCG compatible avec le *benchmarking* rigoureux (Train/Test) :

1. **Inférence par Encodeur Découplé – « La Carte est Fixée »** : Séparation stricte entre l'apprentissage et l'inférence.
 - **Training (Le Graphe construit la Carte)** : Le modèle utilise les connexions entre cellules (graphe) pour structurer l'espace latent.
 - **Test (Projection Individuelle)** : L'encodeur projette chaque cellule individuellement (réseau *feedforward*), sans tenir compte du graphe ni des voisins.
 - *Détail Technique* : Matrice d'adjacence « dummy » (identité) passée en inférence, sans effet mathématique sur les *embeddings*.
2. **Modes de Prédiction (Configurables)** :
 - **Centroid (Défaut)** : Distance au centroïde de *cluster* le plus proche (appris sur le train).
 - **KNN** : Label majoritaire parmi les k plus proches voisins d'entraînement.

4.2.9 Simple Autoencoder

Baseline : Auto-encodeur simple suivi de K-Means.

Fonction de perte MSE :

$$\mathcal{L} = \frac{1}{n \cdot G} \sum_{i=1}^n \sum_{g=1}^G (x_{ig} - \hat{x}_{ig})^2 \quad (96)$$

où n est le nombre de cellules, G le nombre de gènes, x_{ig} la valeur d'entrée et $\hat{x}_{ig}$ la valeur reconstruite.

Hyperparamètres sensibles – Simple Autoencoder

Paramètre	Défaut	Criticité	Description
z_dim	32	Important	Dimension du <i>bottleneck</i> (espace latent).
hidden_layers	256,128	Modéré	Architecture de l'encodeur.
epochs	100	Modéré	Nombre d'epochs. Surveiller la convergence.

Mode d'exécution : Inductif. Baseline simple (Encoder + K-Means) supportant totalement le *transfer learning*.

5 Méthodes de Benchmarking

5.1 Split Train/Val/Test Stratifié

5.1.1 scVI-DCT (Batch Correction / Representation Learning)

*Note : scVI-DCT est implémenté à la fois comme une méthode de **pré-traitement** (correction de batch) et comme un **algorithme autonome** dans le registre, permettant son évaluation directe avec ses propres métriques de clustering (K-Means sur espace latent).*

- **Type** : Probabiliste (VAE) + Metric Learning (Triplet Loss)
- **Inductif** : Oui (via l'inférence variationnelle scVI)
- **Complexité** : Élevée (nécessite obligatoirement GPU/MPS pour des temps de calcul raisonnables)

Description : Adaptation de scVI (Single-cell Variational Inference) intégrant une perte *Domain Class Triplet Loss* (DCT) pour forcer le mélange des batches tout en préservant la séparation des types cellulaires.

Intégration Pipeline : Dans SCRBenchmark, scVI-DCT est implémenté comme un **Algorithme Standalone** (ScVIDCTAlgorithm) via le registre `AlgorithmRegistry`.

- **Entrée** : Requiert strictement des **Raw Counts** (entiers) pour satisfaire l'hypothèse de la distribution Binomiale Négative du VAE. Si des données normalisées sont fournies, le pipeline lève une erreur ou cherche la couche `original_X`.
- **Sortie** : Produit des embeddings latents (Z) qui sont ensuite utilisés pour le clustering (K-Means par défaut).

Détails d'Implémentation (Training Plan) : Le modèle utilise un plan d'entraînement personnalisé `DomainClassTripletLoss_TrainingPlan` qui surcharge la fonction de perte standard (ELBO) :

$$\mathcal{L}_{total} = \mathcal{L}_{ELBO} + \lambda_{DCT}\mathcal{L}_{Triplet} + \lambda_{Corr}\mathcal{L}_{CorrMSE} \quad (97)$$

où $\mathcal{L}_{ELBO}$ est la perte variationnelle standard de scVI, $\mathcal{L}_{Triplet}$ aligne les domaines (batches) en espace latent et $\mathcal{L}_{CorrMSE}$ conserve la structure corrélationnelle ; λ_{DCT} et λ_{Corr} règlent leur contribution relative.

1. Domain Class Triplet Loss ($\mathcal{L}_{Triplet}$) : Optimise la structure de l'espace latent en rapprochant les cellules de même type de différents batches, et en éloignant les cellules de types différents du même batch. Utilise une stratégie de **Hard Sample Mining** (`domain_hard_sample_mining`) :

- *Hard Positive* : La cellule du *même type* mais d'un *batch différent* la plus éloignée.
- *Hard Negative* : La cellule du *même batch* mais d'un *type différent* la plus proche.

2. Correlation MSE ($\mathcal{L}_{CorrMSE}$) : Pour éviter que l'alignement force un "sur-mélange" (over-mixing) qui détruirait la structure biologique fine, on impose que la matrice de corrélation cellule-cellule dans l'espace latent ressemble à celle des covariables d'entrée (PCA par batch).

Technique "Per-Batch PCA" (Conforme scIB-E) : L'implémentation ne calcule pas la corrélation cible sur le dataset global (ce qui inclurait les effets de batch), mais calcule une PCA **indépendamment pour chaque batch** (`compute_per_batch_pca`). Ces composantes principales locales capturent la biologie propre à chaque expérience. Elles sont injectées dans le modèle scVI via des *Continuous Covariates* (`_pca_cov_i`) pour calculer la perte de corrélation :

$$\mathcal{L}_{CorrMSE} = \sum_{b=1}^B \text{MSE}(\text{Corr}(Z_b), \text{Corr}(\text{PCA}(X_b))) \quad (98)$$

où B est le nombre de batches, Z_b les embeddings latents du batch b , X_b ses données d'entrée. La version entraînement somme ce terme par batch ; la validation applique une version globale sur le batch courant.

3. Variante Non-Supervisée (UnsupervisedCorrTrainingPlan) :

Lorsque les labels de types cellulaires ne sont pas disponibles, le plan d'entraînement `UnsupervisedCorrTrainingPlan` remplace la DCT supervisée par un minage de triplets basé sur des **pseudo-labels K-Means**.

- **Pseudo-labels** : À chaque `training_step`, les embeddings latents z sont détachés du graphe de calcul (`z.detach()`) et clusterisés via un K-Means simplifié (`torch.cdist` + assignation itérative, $K=14$, $I=5$ itérations). L'utilisation de `detach()` empêche les gradients de rétro-propager à travers le clustering, qui sert uniquement de signal d'organisation.
- **Hard Mining amélioré** : La sélection des paires suit la même logique (positif = même pseudo-label/batch différent, négatif = pseudo-label différent/même batch), mais les ancres sans paire valide sont **ignorées** plutôt que complétées par des vecteurs nuls.
- **Warmup** : Le poids effectif de la perte triplet augmente linéairement de 0 à λ_{DCT} sur $W=30$ époques, laissant le VAE converger vers une représentation stable avant d'imposer la contrainte triplet.

Comparaison supervisé/non-supervisé : La version supervisée utilise les vrais types cellulaires et applique la DCT dès l'époque 0. La version non-supervisée utilise des pseudo-labels bruités et nécessite un warmup pour stabiliser l'entraînement. Elle est applicable à tout dataset sans annotation.

5.1.2 Proportions par Défaut

Les proportions dépendent de la méthode de split utilisée :

- **GUI (Classical Split)** : 70% Train / 10% Validation / 20% Test (valeurs par défaut, ajustables via sliders).
- **CLI (Benchmark Mode)** : 70% Train / 10% Validation / 20% Test (valeurs par défaut des arguments).
- **Batch-Filtered (Cross-Batch)** : split initial 70/10/20 sur l'ensemble des données, puis *filtrage* des batches pour Train/Val (Test conserve tous les batches, éventuellement équilibré).

Note : Ces valeurs sont configurables via l'interface ou les paramètres CLI.

5.1.3 Stratification

La stratification est **configurable** selon les besoins de l'analyse :

- **Par batch uniquement** (défaut) : Garantit la même proportion de batches dans chaque split. Recommandé pour les expériences cross-batch.

- **Par types cellulaires uniquement** : Garantit la même proportion de labels dans chaque split. Utile si le dataset est mono-batch.
- **Par les deux** : Stratifie simultanément par batch ET par type cellulaire. Peut échouer si certains types cellulaires sont rares dans certains batches.

Note : La stratification par types cellulaires (labels) est **optionnelle** dans l'interface, activée via le sélecteur « Stratify splits by » dans le GUI ou le paramètre `stratify_by_labels` dans le code.

Implémentation Stratification

```

1  from sklearn.model_selection import train_test_split
2
3  def split_stratified(adata, train_ratio=0.7, val_ratio=0.1,
4                      stratify_by='batch'): # or 'labels' or '
5                      both'
6      # Determine stratification groups
7      if stratify_by == 'batch':
8          strat_groups = adata.obs['batch'].astype(str)
9      elif stratify_by == 'labels':
10         strat_groups = adata.obs['labels'].astype(str)
11     else: # both
12         strat_groups = (adata.obs['labels'].astype(str) + '_' +
13                        adata.obs['batch'].astype(str))
14
15     test_ratio = 1.0 - train_ratio - val_ratio # 0.2 by default
16
17     # Premier split: train+val vs test
18     train_val_idx, test_idx = train_test_split(
19         np.arange(len(adata)),
20         test_size=test_ratio,
21         stratify=strat_groups,
22         random_state=42
23     )
24
25     # Second split: train vs val
26     val_relative = val_ratio / (train_ratio + val_ratio)
27     train_idx, val_idx = train_test_split(
28         train_val_idx,
29         test_size=val_relative,
30         stratify=strat_groups[train_val_idx],
31         random_state=42
32     )
33     return adata[train_idx], adata[val_idx], adata[test_idx]

```

5.2 Split par Batch

Pour évaluer la généralisation cross-dataset :

- **Train** : Un ou plusieurs batches spécifiques (ex : inDrop1, inDrop2)
- **Test** : Tous les autres batches

5.3 Balancing des Batches

5.3.1 Problème du Déséquilibre

Les datasets fusionnés ont souvent des batches de tailles très différentes :

Batch	Cellules	%
inDrop3	3,605	22.0%
fluidigmcl	638	3.9%

5.3.2 Méthode de Sous-échantillonnage Stratifié

Algorithm 2 Balancing par Sous-échantillonnage

Require: Dataset $\mathcal{D}$, batches $\{B_1, \dots, B_k\}$

```

1:  $n_{\text{target}} \leftarrow \min_j |B_j|$ 
2: for each batch  $B_j$  do
3:   if  $|B_j| \leq n_{\text{target}}$  then
4:     Conserver toutes les cellules
5:   else
6:     Sous-échantillonner avec stratification par labels
7:      $B'_j \leftarrow \text{StratifiedSample}(B_j, n_{\text{target}})$ 
8:   end if
9: end for
10: return Dataset équilibré  $\mathcal{D}'$ 
```

5.3.3 Stratégies de Réinjection (Usage Avancé)

Pour un benchmark, un jeu de test équilibré entre batches évite qu'un batch majoritaire ne domine les métriques globales. Cependant, l'approche classique (« sous-échantillonnage simple ») jette les données excédentaires, réduisant la taille du jeu d'entraînement.

SCRBenchmark introduit une méthodologie « Core / Surplus » pour résoudre ce dilemme :

- **Étape 1 (Identification)** : Pour chaque batch du jeu de Test, on identifie un **Core** (quota équilibré, ex : 500 cellules/batch) et un **Surplus** (cellules excédentaires).
- **Étape 2 (Distribution)** : Le **Core** constitue le Test Set final (parfaitement équilibré).
- **Étape 3 (Réinjection)** : Le **Surplus** n'est pas jeté mais **réinjecté** dans le Training Set (ou Validation).

Ce mécanisme est contrôlé par le flag `-balance-mode` :

1. **reinject_train** : Les cellules excédentaires du Test sont ajoutées au Train.
2. **reinject_both** : Distribution proportionnelle entre Train et Validation.
3. **standard** (Défaut) : Pas de réinjection (les cellules surnuméraires sont exclues).

Effet : Les algorithmes sont évalués sur un Test set standardisé, tout en apprenant sur 100% de la diversité biologique disponible (Training set non réduit). Les méthodes Deep Learning (scVI, scDeepCluster) utilisent davantage de données d'entraînement dans cette configuration.

6 Modes d'Analyse

Le logiciel propose deux modes d'analyse principaux, chacun adapté à différents cas d'usage.

6.1 Mode Standard

Le mode standard analyse l'ensemble des données sans séparation train/test.

Pipeline :

1. Chargement des données
2. Preprocessing (normalisation, HVG, scaling)
3. Balancement optionnel des batches (via `-balance-batches-standard`)
4. Entraînement de l'algorithme : `fit(data, labels)`
5. Prédiction sur les mêmes données : `predict(data)`
6. Calcul des métriques vs labels réels

Cas d'usage :

- Exploration et découverte de clusters
- Analyse d'un seul dataset
- Comparaison rapide d'algorithmes

Note

En mode standard, les métriques reflètent la qualité du clustering sur les données d'entraînement. Ce n'est pas une mesure de généralisation.

6.2 Mode Benchmark

Le mode benchmark sépare les données en train/test pour évaluer la capacité de généralisation.

Pipeline :

1. Chargement des données
2. Split stratifié : ratios configurables (défauts GUI/CLI décrits ci-dessus)
3. Preprocessing sur train, appliqué à val/test
4. Entraînement : `fit(train_data, train_labels)`
5. Validation : early stopping / scheduler de LR si supporté
6. Prédiction sur test : `predict(test_data)`
7. Métriques sur train, val et test séparément

Cas d'usage :

- Évaluation de la généralisation
- Détection du sur-apprentissage
- Benchmarking scientifique rigoureux
- Simulation de déploiement en production

Note sur les Algorithmes Transductifs (Fallback)

Ce mode est conçu pour le **transfer learning** (inductif). Cependant, pour permettre une comparaison exhaustive, les algorithmes transductifs (scMUG, scHNTL) ne sont pas bloqués dans le GUI/CLI mais exécutés en **Mode Fallback Transductif** :

1. Le pipeline divise les données (Train/Test) pour les algorithmes inductifs.
2. Pour les algorithmes transductifs, les données sont temporairement réunifiées (Train + Test).
3. L'algorithme apprend sur l'ensemble complet, et `predict()` est évalué sur ce même ensemble.
4. Les métriques « Test » reportées ne mesurent **pas** la généralisation *out-of-sample* : ce sont des métriques transductives calculées sur l'ensemble complet.

Attention : Dans le cœur du pipeline (**AnalysisRunner**), un vrai scénario out-of-sample lève une **ValueError** pour ces algorithmes. Le GUI/CLI contournent ce cas via le fallback décrit ci-dessus.

7 Optimisation Automatique des Hyperparamètres (AutoML)

SCRBenchmark intègre un module d'optimisation automatique des hyperparamètres en amont de l'évaluation finale. Cette fonctionnalité permet à chaque algorithme d'être évalué avec des hyperparamètres adaptés au jeu de données.

7.1 Workflow d'Optimisation

Le processus s'intègre en amont de l'évaluation finale. Pour éviter tout data leakage scientifique, l'optimisation doit être réalisée dans une session dédiée. En **Mode Benchmark**, l'optimisation des hyperparamètres est **désactivée** par défaut pour la mesure de performance. Le split de Validation est alors réservé exclusivement :

- Au **Early Stopping** : Arrêter l'entraînement si la loss de reconstruction stagne.
- Au **Learning Rate Scheduler** : Réduire le LR sur plateau.

Pour trouver les meilleurs hyperparamètres, l'utilisateur doit utiliser le module dédié « Hyperparameter Search » (GUI) ou lancer des expériences séparées, puis reporter les meilleures valeurs pour l'évaluation finale sur le Test set. Cela évite le biais d'optimisation (*overfitting* sur le set de validation) lors de la mesure de généralisation.

8 Transfer Learning pour l'Évaluation

8.1 Principe

Pour évaluer la généralisation, les modèles sont entraînés sur le training set puis utilisés pour prédire sur le test set **sans réentraînement**.

$$\text{Train} : X_{\text{train}} \xrightarrow{\text{fit}} \theta^* \xrightarrow{\text{encode}} Z_{\text{train}} \xrightarrow{\text{cluster}} \mu_1, \dots, \mu_k \quad (99)$$

$$\text{Test} : X_{\text{test}} \xrightarrow{\text{encode}(\theta^*)} Z_{\text{test}} \xrightarrow{\text{assign}(\mu)} \hat{y}_{\text{test}} \quad (100)$$

8.2 Compatibilité des Algorithmes

Algorithme	Graphe ?	Transfer Learning	Méthode
PCA + K-Means	Non	Complet	PCA.transform + KMeans.predict
PCA + Leiden	Non*	Complet	PCA.transform + KNN
SimpleAutoencoder	Non	Complet	Encoder + KMeans.predict
scDeepCluster	Non	Complet	Encoder + soft_assign
scMAE	Non	Complet	Preproc + Encoder + KMeans.predict
scNAME	Non	Complet	Encoder + distance aux centres
scVI-DCT	Non	Complet	scVI latent + KMeans.predict
scCDCG	Oui	Complet	Encoder + Centroid/KNN
scMUG	Oui	Non supporté	Spectral clustering
scHNTL	Oui	Non supporté	Graph Attention

TABLE 3 – Compatibilité des algorithmes avec le *transfer learning*. Les algorithmes basés sur des graphes globaux (scMUG, scHNTL) ne supportent pas la prédiction *out-of-sample*.

* *PCA + Leiden* utilise un graphe KNN mais celui-ci est reconstruit localement. Pour le *transfer learning*, un classifieur KNN est entraîné sur les labels Leiden.

8.3 Algorithmes Non Basés sur Graphes

Ces algorithmes peuvent facilement prédire sur de nouvelles données :

1. **Encoder** : Applique les poids du réseau entraîné aux nouvelles données
2. **Embeddings** : Obtient les représentations latentes
3. **Assignment** : Utilise les centres de clusters appris

8.3.1 Pour K-Means

$$\hat{y}_i = \arg \min_j \|z_i - \mu_j\|^2 \quad (101)$$

8.3.2 Pour scDeepCluster/scNAME

$$\hat{y}_i = \arg \max_j q_{ij} \quad \text{où} \quad q_{ij} = \text{soft_assign}(z_i, \mu_j) \quad (102)$$

Implémentation Predict

```

1 def predict(self, X_new):
2     self.model.eval()
3     with torch.no_grad():
4         # Encoder avec poids figes
5         z_new = self.model.encode(X_new)
6         # Assigner aux clusters
7         q = self.model.soft_assign(z_new)
8         labels = q.argmax(dim=1)
9     return labels.numpy()

```

8.4 Algorithmes Basés sur Graphes : Limitations

Les algorithmes basés sur graphes (scMUG, scHNTL) ne supportent pas le transfer learning pour les raisons suivantes :

8.4.1 Problème fondamental

L'encodeur utilise la **structure du graphe** pour propager l'information entre cellules voisines.

$$h'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j \right) \quad (103)$$

Dans l'implémentation actuelle de ces méthodes, le processus est **transductif** : le graphe est construit sur l'ensemble des données disponibles lors de l'entraînement. L'inférence sur de nouvelles cellules isolées (inductive learning) n'est pas prise en charge nativement sans reconstruire le graphe ou ré-entraîner le modèle.

8.4.2 Conséquences pratiques

Note technique

Ces algorithmes ne supportent pas le transfer learning strict. Ils sont marqués comme `supports_out_of_sample=False`. Dans le cœur du pipeline (`AnalysisRunner`), un vrai scénario out-of-sample déclenche une `ValueError`. Le GUI/CLI contournent ce cas en mode benchmark via un **fallback transductif** (entraînement et évaluation sur l'ensemble complet).

Alternatives pour ces algorithmes :

1. Utiliser le mode Standard (pas de split)
2. Utiliser la cross-validation avec ré-entraînement complet
3. Choisir un algorithme non basé sur graphe pour les benchmarks

9 Évitement du Data Leakage

9.1 Preprocessing

Le preprocessing (normalisation, scaling) doit utiliser uniquement les statistiques du training set.

Preprocessing Sans Leakage

```

1  class BenchmarkPreprocessor:
2      def fit(self, X_train):
3          self.mean_ = X_train.mean(axis=0)
4          self.std_ = X_train.std(axis=0)
5          self.std_[self.std_ == 0] = 1  # Eviter div/0
6          return self
7
8      def transform(self, X):
9          return (X - self.mean_) / self.std_
10
11     def fit_transform(self, X_train):
12         return self.fit(X_train).transform(X_train)
13
14     # Usage
15     preprocessor = BenchmarkPreprocessor()
16     X_train_scaled = preprocessor.fit_transform(X_train)
17     X_test_scaled = preprocessor.transform(X_test)  # Utilise stats
               train

```

9.2 Sélection des HVG

La sélection des gènes hautement variables (HVG) conditionne la reproductibilité du benchmarking. Le logiciel propose trois stratégies :

9.2.1 Stratégie 1 : Train Only (Défaut, Recommandé)

Les HVG sont sélectionnés uniquement sur les données d'entraînement. C'est la stratégie la plus rigoureuse scientifiquement car elle évite tout data leakage.

```

1  # Selection HVG sur le train uniquement
2  sc.pp.highly_variable_genes(adata_train, n_top_genes=5000)
3  hvg_genes = adata_train.var_names[adata_train.var['highly_variable']]
4
5  # Appliquer le meme subset au test
6  adata_test = adata_test[:, hvg_genes]

```

Avantages :

- Aucun data leakage
- Simule un vrai scenario de deployment

Inconvénients :

- Certains gènes importants présents uniquement dans le test peuvent être exclus

9.2.2 Stratégie 2 : All Data

Les HVG sont sélectionnés sur l'ensemble des données (train + test) avant le split. Le preprocessing est ensuite fait sur le train et appliqué au test.

```

1  # Selection HVG sur toutes les donnees
2  sc.pp.highly_variable_genes(adata_full, n_top_genes=5000)
3  hvg_genes = adata_full.var_names[adata_full.var['highly_variable']]
4
5  # Puis split et preprocessing
6  adata = adata_full[:, hvg_genes]

```

```

7 train, test = split(adata)
8 preprocessor.fit(train)
9 train = preprocessor.transform(train)
10 test = preprocessor.transform(test)

```

Attention : Data Leakage

Cette stratégie introduit un léger data leakage car la sélection des HVG utilise des informations du test set. À utiliser principalement pour comparaison avec la littérature existante qui suit souvent cette approche.

Note d'implémentation : Cette stratégie est disponible via la CLI/configuration. En mode benchmark dans le GUI, l'option **All Data** est masquée pour éviter toute fuite d'information par défaut.

9.2.3 Stratégie 3 : Per-Batch Union

Pour les données multi-batch, sélectionne les top-N HVG dans chaque batch séparément, puis prend l'union de tous ces gènes.

```

1 all_hvg = set()
2 for batch in batches:
3     adata_batch = adata[adata.obs['batch'] == batch]
4     sc.pp.highly_variable_genes(adata_batch, n_top_genes=n_top)
5     batch_hvg = adata_batch.var_names[adata_batch.var['
        highly_variable']]
6     all_hvg.update(batch_hvg)
7
8 # Union des HVG de tous les batches
9 adata = adata[:, list(all_hvg)]

```

Avantages :

- Capture les gènes variables spécifiques à chaque batch
- Utile pour les données très hétérogènes

Inconvénients :

- Produit généralement plus de gènes que `n_top_genes`
- Peut introduire du bruit si certains batches ont des gènes variables non biologiques

10 Métriques par Groupe

Pour identifier les sources d'erreurs, les métriques sont calculées séparément par batch.

Note sur l'Accuracy (ACC) par Batch : Contrairement au NMI qui est invariant par permutation, l'ACC nécessite un alignement entre clusters et labels. SCRbenchmark utilise un **Alignement Global** : le mapping optimal est déterminé sur l'ensemble des données de test combinées. Ce mapping fixe est ensuite appliqué à chaque batch pour calculer son accuracy locale. Cela diffère de l'alignement local (calculé batch par batch) qui masquerait les incohérences d'assignation du modèle entre les différents lots.

$$\text{NMI}_{B_j} = \text{NMI}(y_{\text{true}}^{B_j}, y_{\text{pred}}^{B_j}) \quad (104)$$

Implémentation

```

1  def compute_metrics_by_group(y_true, y_pred, groups):
2      results = {}
3      for group in np.unique(groups):
4          mask = groups == group
5          results[group] = {
6              'NMI': normalized_mutual_info_score(y_true[mask],
7              y_pred[mask]),
8              'ARI': adjusted_rand_score(y_true[mask], y_pred[mask]
9              ),
10             'ACC': cluster_acc(y_true[mask], y_pred[mask]),
11             'n_cells': mask.sum()
12         }
13     return results

```

11 Architecture du Logiciel

11.1 Structure des Fichiers

```

SCRBenchmark/
|-- scrbenchmark          # Wrapper CLI (script bash)
|-- src/
|   +-- scrbenchmark/
|       |-- app.py        # Main Streamlit application
|       |-- cli.py        # Command-line interface
|       |-- algorithms/   # Implementations des algorithmes
|       |-- core/         # Composants systeme
|       |-- gui/          # Modules de l'interface graphique
|       +-- utils/        # Fonctions utilitaires
|-- external/             # Code source original des auteurs
|-- tests/                # Tests unitaires et d'integration
|-- scripts/              # Scripts d'exploration et fusion
+-- results_CLI/          # Resultats des analyses CLI

```

11.2 Accélération Matérielle et Support MPS

Le logiciel détecte et utilise automatiquement l'accélération matérielle disponible.

- **CUDA (NVIDIA)** : Support standard pour tous les algorithmes Deep Learning.
- **MPS (Apple Silicon - M1/M2/M3/M4)** : Support natif optimisé via Metal Performance Shaders.
 - *Optimisation* : Conversion automatique des tenseurs en `float32` pour éviter les erreurs de précision (NaN) fréquentes lors des calculs de gradients sur MPS.
 - *Stabilité* : `gradient_clipping` (valeur 0.5) est appliqué dans le plan d'entraînement scVI-DCT pour éviter l'explosion des gradients sur MPS. Ce n'est pas un réglage global pour tous les algorithmes.
- **CPU** : Fallback automatique si aucun GPU n'est détecté.

11.2.1 Reproductibilité Cross-Platform : `force_float32`

Problème : Par défaut, le logiciel utilise `float64` sur CPU/CUDA et `float32` sur MPS (car MPS ne supporte pas `float64`). Cela peut créer des différences de résultats entre plateformes.

Solution : L'option `force_float32` permet de forcer l'utilisation de `float32` sur toutes les plateformes.

Usage de `force_float32`

```

1  # Dans l'interface GUI ou CLI
2  params = {
3      'force_float32': True, # Forcer float32 partout
4      ...
5  }
6
7  # Comportement interne (scDeepCluster)
8  if device == "mps" or force_float32:
9      np_dtype = np.float32
10     torch_dtype = torch.float32
11 else:
12     np_dtype = np.float64
13     torch_dtype = torch.float64

```

Recommandations :

- `force_float32=False` (défaut) : Précision maximale sur CPU/CUDA, résultats potentiellement différents de MPS.
- `force_float32=True` : Résultats identiques entre plateformes, légère perte de précision numérique sur CPU/CUDA.

Cas d'usage : Activer `force_float32` si vous devez comparer des résultats entre un serveur GPU NVIDIA et un Mac avec Apple Silicon.

11.3 Interface CLI

```

1  # Mode standard
2  ./scribenchmark run --data data.h5ad --algorithms pca_kmeans,
   scdeepcluster
3
4  # Mode benchmark
5  ./scribenchmark run \
6      --data data.h5ad \
7      --algorithms pca_kmeans,scdeepcluster \
8      --benchmark-mode \
9      --train-ratio 0.7 \
10     --val-ratio 0.1 \
11     --test-ratio 0.2 \
12     --stratify-by tech \
13     --batch-key batch \
14     --exclude-batches smarter \
15     --hvg-strategy train_only \
16     --output results_CLI/benchmark

```

12 Hyperparamètres - Description Détaillée

Cette section fournit une description de chaque hyperparamètre et son rôle. *Note : Les plages de valeurs et recommandations présentées ci-dessous sont issues de la littérature scientifique associée à chaque algorithme et d'observations empiriques courantes en scRNA-seq.*

12.1 Hyperparamètres Principaux (Architecture et Clustering)

12.1.1 n_clusters (k)

Définition : Nombre de clusters à former, correspondant au nombre de types cellulaires attendus dans le dataset.

Rôle mathématique :

- Définit le nombre de centroïdes $\{\mu_1, \dots, \mu_k\}$ dans l'espace latent
- Pour scDeepCluster : taille de la couche de clustering $\in \mathbb{R}^{k \times d}$ où d est la dimension latente
- Pour GMM : nombre de composantes Gaussiennes $\sum_{j=1}^k \pi_j \mathcal{N}(\mu_j, \Sigma_j)$

Impact précis :

- $k < k_{\text{réel}}$: Fusion de types cellulaires distincts, perte d'information
- $k > k_{\text{réel}}$: Fragmentation artificielle, sur-segmentation des clusters
- Affecte directement NMI et ARI car ces métriques comparent aux labels réels

Estimation : Utiliser la silhouette ou le BIC sur un range $k \in [\frac{k_{\text{attendu}}}{2}, 2 \times k_{\text{attendu}}]$.

12.1.2 use_ground_truth_k (Mode Oracle)

Définition : Paramètre booléen explicite permettant d'utiliser le nombre de clusters k déterminé à partir des labels ground truth.

AVERTISSEMENT : Mode Oracle

Ce paramètre constitue une fuite d'information du test set ! L'utilisation de `use_ground_truth_k=True` signifie que le modèle reçoit une information (k) qu'il n'aurait pas en conditions réelles de déploiement.

Conséquences scientifiques :

- Les résultats obtenus avec ce mode NE SONT PAS comparables aux méthodes qui estiment k de manière indépendante
- Ce mode ne doit être utilisé QUE pour :
 - Débuggage et validation du code
 - Reproduction de résultats publiés qui ont utilisé cette approche
 - Analyse de sensibilité pour évaluer l'impact de l'estimation de k

Valeur par défaut : False (désactivé)

Comportement :

- `use_ground_truth_k=False` (recommandé) : Le nombre de clusters doit être spécifié manuellement via `n_clusters` ou estimé automatiquement par l'algorithme (ex : Leiden pour scDeepCluster).
- `use_ground_truth_k=True` : $k = |\text{unique}(y_{\text{true}})|$ est calculé à partir des labels ground truth.

Algorithmes concernés : PCA+K-Means, scDeepCluster, et tout algorithme nécessitant k en entrée.

12.1.3 z_dim (Dimension latente)

Définition : Dimension de l'espace de représentation compressé $z \in \mathbb{R}^d$ en sortie de l'encodeur.

Rôle mathématique :

$$x \in \mathbb{R}^G \xrightarrow{\text{Encoder}} z \in \mathbb{R}^d \xrightarrow{\text{Decoder}} \hat{x} \in \mathbb{R}^G \quad (105)$$

où $G \approx 5000$ (gènes) et $d \in [16, 64]$ typiquement.

Impact précis :

- **Trop petit** ($d < 10$) : Goulot d'étranglement trop sévère, perte d'information biologique. Le modèle ne peut pas capturer la variabilité entre types cellulaires.
- **Trop grand** ($d > 100$) : Pas assez de compression, le modèle ne généralise pas. Risque d'overfitting car le latent conserve du bruit technique.
- **Optimal** : d tel que la reconstruction soit bonne ET que l'espace latent soit suffisamment structuré pour le clustering. En pratique, $d \in [20, 50]$ pour scRNA-seq.

12.1.4 gamma / lambda (Poids de la loss de clustering)

Définition : Coefficient qui pondère l'importance relative de la perte de clustering par rapport à la reconstruction.

Rôle mathématique :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \gamma \cdot \mathcal{L}_{\text{cluster}} \quad (106)$$

Impact précis :

- $\gamma = 0$: Pur auto-encodeur, pas de signal de clustering. Les représentations ne sont pas guidées vers une structure clusterisée.
- $\gamma \ll 1$: Reconstruction dominante. Bonne reconstruction mais clusters potentiellement mal séparés.
- $\gamma \gg 1$: Clustering dominant. Clusters bien séparés mais reconstruction dégradée, risque d'effondrement des représentations.
- **Balance optimale** : $\gamma \in [0.1, 10]$ selon l'algorithme. Pour scDeepCluster : $\gamma = 1$ par défaut.

12.1.5 temperature (τ)

Définition : Paramètre de température dans les distributions softmax, contrôlant la « dureté » des assignations probabilistes.

Rôle mathématique :

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (107)$$

Impact précis :

- $\tau \rightarrow 0$: Distribution devient one-hot (assignation dure). Chaque cellule est assignée avec probabilité 1 à un seul cluster.
- $\tau \rightarrow \infty$: Distribution uniforme. Aucune discrimination entre les clusters.
- **Pour contrastive learning** : $\tau \in [0.07, 0.5]$. τ bas exige des représentations très distinctes pour les paires négatives.
- **Pour spectral softmax** : $\tau \in [0.1, 1.0]$. Affecte la transition entre clusters flous et durs.

12.2 Hyperparamètres d'Architecture Neuronale

12.2.1 encoder_layers / hidden_dims

Définition : Liste des dimensions des couches cachées de l'encodeur, ex : « 512,256,64 » signifie 3 couches de 512, 256, puis 64 neurones.

Interface : L'architecture est saisie via trois champs standardisés : **Encodeur**, **Espace latent**, **Decodeur**, par exemple : [256,64] ; [32] ; [256,64]. Pour les modèles à decodeur symétrique (ou fixe), le champ decodeur peut être ignoré.

Rôle mathématique :

$$h^{(1)} = \sigma(W_1 x + b_1), \quad h^{(2)} = \sigma(W_2 h^{(1)} + b_2), \quad z = W_3 h^{(2)} + b_3 \quad (108)$$

où $W_1 \in \mathbb{R}^{512 \times G}$, $W_2 \in \mathbb{R}^{256 \times 512}$, $W_3 \in \mathbb{R}^{d \times 256}$.

Impact précis :

- **Plus large** (512+) : Capacité à capturer des patterns complexes. Nécessite plus de données pour éviter l'overfitting.
- **Plus profond** (4+ couches) : Hiérarchie de représentations. Risque de vanishing gradients sans skip connections.
- **Pyramidal décroissant** (512→256→64) : Compression progressive, standard pour autoencodeurs.
- **Règle pratique** : Première couche $\leq G/4$, réduction par facteur 2-4 à chaque couche.

12.2.2 n_heads (Attention multi-tête)

Définition : Nombre de têtes d'attention parallèles dans un Graph Attention Network.

Rôle mathématique :

$$h'_i = \left\|_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} h_j \right) \right. \quad (109)$$

où $\parallel$ dénote la concaténation et K est le nombre de têtes.

Impact précis :

- **Plus de têtes** : Capture différents types de relations cellule-cellule simultanément. Chaque tête peut apprendre un pattern d'attention différent.
- **Trade-off** : K têtes avec dimension d/K chacune (si on moyenne) ou $K \times d$ (si on concatène). Plus de paramètres.
- **Typique** : $K \in [1, 8]$. Au-delà de 8, gains marginaux.

12.2.3 dropout

Définition : Probabilité de mettre à zéro chaque activation pendant l'entraînement.

Rôle mathématique :

$$\tilde{h}_i = \frac{m_i \cdot h_i}{1 - p} \quad \text{où } m_i \sim \text{Bernoulli}(1 - p) \quad (110)$$

Impact précis :

- **Régularisation** : Force le réseau à ne pas dépendre de neurones spécifiques. Équivalent à un ensemble de sous-réseaux.
- $p = 0$: Pas de régularisation, risque d'overfitting.
- $p = 0.5$: Standard pour les couches cachées.
- $p > 0.7$: Trop agressif, sous-apprentissage.
- **Pour GAT** : $p \in [0.3, 0.6]$ sur les features et l'attention.

12.3 Hyperparamètres d'Entraînement

12.3.1 pretrain_epochs

Définition : Nombre d'époques de pré-entraînement de l'auto-encodeur AVANT d'activer la perte de clustering.

Rôle :

1. Entraîner l'AE seul pendant `pretrain_epochs` avec $\mathcal{L} = \mathcal{L}_{\text{recon}}$
2. Initialiser les centroïdes avec K-Means sur les z appris
3. Fine-tuner avec $\mathcal{L} = \mathcal{L}_{\text{recon}} + \gamma \mathcal{L}_{\text{cluster}}$

Impact précis :

- **Trop peu** (< 50) : Représentations initiales de mauvaise qualité. Les centroïdes K-Means sont mal positionnés. Le clustering démarre avec une mauvaise initialisation.
- **Suffisant** (100-300) : L'espace latent a une structure significative. Les clusters initiaux correspondent approximativement aux types cellulaires.
- **Trop** (> 500) : Temps de calcul gaspillé, pas de gain car la loss de reconstruction converge.

12.3.2 learning_rate

Définition : Pas de mise à jour des poids à chaque itération de gradient descent.

Rôle mathématique :

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L} \quad (111)$$

Impact précis :

- η **trop grand** (> 0.01) : Oscillations autour du minimum, instabilité. Peut diverger.
- η **trop petit** ($< 10^{-5}$) : Convergence extrêmement lente. Peut rester bloqué dans des minima locaux.
- **Optimal pour Adam** : $\eta \in [10^{-4}, 10^{-3}]$
- **Scheduling** : Réduire η après plateau de validation (`ReduceLROnPlateau`)

12.3.3 batch_size

Définition : Nombre d'échantillons (cellules) utilisés pour calculer un gradient avant mise à jour.

Impact précis :

- **Petit** (16-64) : Gradient bruité, effet de régularisation implicite. Convergence lente mais peut trouver de meilleurs minima.
- **Grand** (256-1024) : Gradient stable, convergence rapide. Nécessite plus de mémoire GPU. Peut converger vers des minima plus plats (meilleure généralisation).
- **Règle** : Plus grand possible selon la mémoire disponible. Adapter le learning rate ($\eta_{\text{effective}} = \eta \times \text{batch_size}/256$).

12.4 Hyperparamètres de Graphe

12.4.1 knn_k / leiden_neighbors

Définition : Nombre de voisins les plus proches pour construire le graphe cellulaire.

Construction du graphe :

$$A_{ij} = \begin{cases} 1 & \text{si } j \in \text{kNN}(i) \text{ ou } i \in \text{kNN}(j) \\ 0 & \text{sinon} \end{cases} \quad (112)$$

Impact précis :

- **Petit** ($k = 5$) : Graphe épars, clusters très séparés. Risque de fragmenter un type cellulaire en plusieurs composantes.
- **Grand** ($k = 50$) : Graphe dense, clusters connectés entre eux. Risque de fusionner des types cellulaires proches.
- **Règle** : $k \approx \sqrt{n}$ ou $k \in [10, 30]$ pour scRNA-seq typique.

12.4.2 leiden_resolution (γ)

Définition : Paramètre de résolution dans l'optimisation de la modularité.

Rôle mathématique :

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (113)$$

Impact précis :

- $\gamma < 0.5$: Favorise de grands clusters. Détecte les structures de haut niveau (ex : immunitaire vs épithélial).
- $\gamma = 1.0$: Modularité standard.
- $\gamma > 1.5$: Favorise de petits clusters. Détecte les sous-types fins (ex : CD4+ naïves vs mémoire).
- **Calibration** : Tester $\gamma \in [0.1, 0.3, 0.5, 0.8, 1.0, 1.5, 2.0]$ et choisir selon le nombre de clusters attendu.

12.5 Hyperparamètres Spécifiques

12.5.1 masking_rate (scMAE)

Définition : Proportion de valeurs d'expression génique perturbées par corruption shuffle (mélangées avec des valeurs d'autres cellules) avant encodage.

Rôle : Créer une tâche de prétexte difficile (estimation du masque + reconstruction) pour apprendre des représentations robustes capturant les corrélations entre gènes.

Impact précis :

- $\rho = 0.1$: Trop facile. Le modèle peut reconstruire en s'appuyant principalement sur les entrées non perturbées.
- $\rho = 0.4$: Optimal (défaut). La corruption par shuffle est moins destructive que le masquage à zéro car les valeurs restent dans la distribution des données.
- $\rho > 0.5$: Difficile. Risque que la reconstruction et l'estimation du masque deviennent trop difficiles, dégradant la qualité des représentations.
- **Note** : La corruption shuffle (échange avec une autre cellule) est fondamentalement différente du masquage à zéro utilisé dans le MAE vision.

12.5.2 alpha (DEC/scDeepCluster)

Définition : Degrés de liberté de la distribution Student-t pour le soft assignment.

Rôle mathématique :

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2 / \alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'} (\cdot)^{-\frac{\alpha+1}{2}}} \quad (114)$$

Impact précis :

- $\alpha = 1$: Distribution de Cauchy. Queues lourdes, tolère les outliers.
- $\alpha \rightarrow \infty$: Converge vers une Gaussienne. Moins tolérant aux outliers.
- **Défaut** : $\alpha = 1$ recommandé (papier DEC original).

12.6 Visualisation et Interprétation des Résultats

Cette section explique comment interpréter les graphiques générés automatiquement par le logiciel, ainsi que les fonctions sous-jacentes.

12.6.1 Figures de l'onglet Results Explorer (src/scrbenchmark/gui/results_explorer.py)

L'onglet Results Explorer expose les types de visualisations définis dans `FIGURE_TYPES`. La table ci-dessous décrit ce que fait chaque figure dans l'implémentation actuelle.

Clé UI	Sortie	Calcul / contenu
<code>algorithm_comparison</code>	Barplot groupé	Compare <code>metric_mean</code> $\pm$ <code>metric_std</code> par algorithme et condition (agrégats issus de <code>agg_df</code>). Option CLD possible si suffisamment de répétitions.
<code>metrics_heatmap</code>	Heatmap algorithme $\times$ condition	Affiche la matrice pivotée de <code>metric_mean</code> . Échelle [0,1] si toutes les valeurs y appartiennent, sinon bornes auto min/max.
<code>generalization_gap</code>	Boxplot du gap	En mode <code>benchmark_detailed</code> , calcule pour chaque run : $\Delta = \text{train}_m - \text{test}_m$, puis boxplot par algorithme.
<code>generalization_gap_heatmap</code>	Heatmap du gap	Même $\Delta = \text{train}_m - \text{test}_m$, agrégé en moyenne et écart-type par (algorithme, condition), affiché en heatmap centrée sur 0.
<code>batch_generalization</code>	Matrice inter-batch (par algorithme)	Lit <code>test_by_group</code> , extrait ACC par batch test, puis moyenne par couple (batch entraînement, batch test). Ce n'est pas une différence train-test. Cases <code>train=test</code> encadrées en bleu.
<code>test_metrics_by_batch</code>	3 heatmaps (NMI/ARI/ACC)	Parse <code>test_by_group</code> , agrège NMI/ARI/ACC par (algorithme, batch test), puis affiche trois matrices séparées (une par métrique).
<code>saved_umap_gallery</code>	Galerie d'images UMAP	Ne recalcule rien : relit les PNG déjà exportés (<code>figures/umap_*</code>). Supporte split <code>train/test/val/full</code> .
<code>umap_diagnostic</code>	Figure UMAP diagnostique 2x2	Reconstruit une vue 2x2 à partir des labels sauvegardés + <code>h5ad</code> . Choix des embeddings : <code>X_umap</code> puis <code>X_pca</code> , sinon <code>X</code> tronqué. Affiche vrai/prédit/batch/erreurs via <code>viz_utils.plot_umap_diagnostic</code> .
<code>celltype_errors</code>	Heatmap erreur par type cellulaire	Source prioritaire : <code>error_analysis.error_by_celltype</code> . Fallback mode standard : <code>ClassWise</code> avec taux d'erreur = $1 - \text{Recall}$.
<code>celltype_errors_by_batch</code>	Heatmaps erreur type $\times$ batch	Utilise <code>error_analysis.error_by_celltype_by_group</code> , puis moyenne les taux d'erreur par (type, batch), une heatmap par algorithme.
<code>error_rate_by_batch</code>	Heatmap erreur par batch	Priorité : <code>error_analysis.error_by_group.error_rate</code> . Fallback : $1 - \text{ACC}$ depuis <code>test_by_group</code> . Batches triés du plus difficile au plus facile.
<code>confusion_patterns</code>	Barplot Top confusions	Agrège les paires <code>true</code> $\rightarrow$ <code>predicted</code> depuis <code>error_analysis.top_confusion_pairs</code> , puis affiche le Top-15 des confusions les plus fréquentes.
<code>confusion_matrix_detailed</code>	Matrices de confusion multi-algos	Construit la confusion globale par algorithme à partir des CSV labels (ou fallback colonnes CSV), avec alignement hongrois si espace de labels différent. Affiche proportions + compte moyen/run + Macro F1, précision, Balanced ACC, Rare-ACC.
<code>confusion_matrix_by_batch</code>	Matrices de confusion par batch	Même principe que ci-dessus, mais séparé par batch test. Requiert la colonne <code>batch</code> dans les CSV de labels (sinon figure indisponible).
<code>metrics_by_celltype_table</code>	Tableau F1/Précision/Rappel par type	Calcule <code>precision_recall_fscore_support</code> après éventuel alignement hongrois des labels prédits, puis produit un tableau détaillé par algorithme, condition et type cellulaire.
<code>runtime_comparison</code>	Barplot horizontal runtimes	Moyenne <code>runtime_mean</code> (et <code>runtime_std</code>) par algorithme sur les conditions sélectionnées; affiche barres d'erreur et annotation en secondes.

Clé UI	Sortie	Calcul / contenu
<code>summary_table</code>	Tableau récapitulatif	Génère une table des métriques agrégées au format $\mu \pm \sigma$ par algorithme et condition (pas une figure graphique).
<code>statistical_test</code>	Tableau de tests statistiques	Test Mann-Whitney pairwise (conditions ou algorithmes), tailles d'effet (rank-biserial), correction multiple (FDR/Bonferroni si <code>statsmodels</code>).
<code>train_vs_test</code>	Scatter train vs test (5 métriques)	Pour NMI/ARI/ACC/F1_Macro/BalancedACC, place (train, test) par algorithme/condition ; diagonale = généralisation idéale, zone au-dessus = overfitting.
<code>generalization_gap_combined</code>	5 boxplots de gap	Pour les 5 métriques ci-dessus, calcule $\Delta = \text{train} - \text{test}$, trie les algorithmes par gap moyen croissant, ajoute seuils 0/5%/10% et points individuels.
<code>publication_export</code>	Export texte (LaTeX/Markdown)	Produit un tableau prêt publication avec meilleures valeurs mises en gras (pas une figure graphique).
<code>loss_curves</code>	Galerie courbes de perte	Cherche les historiques de perte (JSON dans <code>loss_history/</code> ou PNG dans <code>figures/</code>) pour chaque algorithme et condition. Affiche les courbes d'entraînement par phase (pré-entraînement, fine-tuning) avec identification des composantes de perte (reconstruction, triplet, KL, etc.).
<code>umap_evolution</code>	Galerie évolution UMAP	Relit les PNG <code>umap_evolution_<algo>_run*.png</code> exportés pendant l'entraînement. Chaque sous-figure montre la projection UMAP des embeddings à un epoch donné, colorée par type cellulaire réel. Permet de visualiser la séparation progressive des clusters au fil de l'entraînement.
<code>batch_composition</code>	Barres empilées + heatmaps	Pour chaque condition, construit la composition en batches des splits train et test (moyennée sur les runs). Affiche : (1) barres horizontales empilées montrant les proportions de chaque batch par split, (2) heatmaps type cellulaire $\times$ batch pour identifier les biais (ex : un type cellulaire présent dans un seul batch du train), (3) tableau des effectifs exacts. Requiert la colonne <code>batch</code> dans les CSV de labels.

Remarques d'implémentation.

- Les visualisations dépendant de `train/test` (`generalization_gap*`, `train_vs_test`, `batch_generalization`, `test_metrics_by_batch`) nécessitent des résultats `benchmark_detailed`.
- Les figures d'erreur par batch/type exploitent d'abord `error_analysis` ; en son absence, des fallbacks sont appliqués (1 – ACC, `ClassWise`).
- Les matrices de confusion réalignent les clusters prédits vers les labels vrais via l'algorithme hongrois quand les espaces de labels diffèrent.
- Si l'option `split_figures_by_condition` est activée dans l'interface, la même fonction de tracé est exécutée séparément pour chaque condition sélectionnée.
- `loss_curves` et `umap_evolution` reposent sur des fichiers exportés pendant l'entraînement (JSON/PNG) ; si absents, la figure est vide.
- `batch_composition` requiert la colonne `batch` dans les CSV de labels ; en son absence, un message d'indisponibilité est affiché.

12.6.2 Radar Chart (Profil de Performance)

Visualisation

Graphique polaire comparant simultanément plusieurs algorithmes sur 4 métriques (NMI, ARI, ACC, Silhouette).

Génération : `viz.plot_radar_chart(results.summary)` dans `utils/visualization.py`.

Interprétation :

- Chaque axe représente une métrique (échelle [0,1]).
- L'aire couverte par un algorithme reflète sa performance globale.
- Un polygone couvrant une grande aire indique des scores élevés sur l'ensemble des métriques.
- Un polygone irrégulier (élevé sur NMI, faible sur Silhouette) indique des performances hétérogènes selon les métriques.

12.6.3 Matrice de Confusion (Clusters vs Labels)

Visualisation

Heatmap montrant la correspondance entre les clusters prédits (colonnes) et les types cellulaires réels (lignes).

Génération : `viz.plot_confusion_matrix_multi(results.results, true_labels)` dans `utils/visualization.py`.

Méthodologie :

- Construit une matrice de contingence via `sklearn.metrics.cluster.contingency_matrix`.
- Réordonne les colonnes via l'**algorithme hongrois** (`linear_sum_assignment`) pour maximiser la correspondance.
- Normalise par ligne (chaque type cellulaire somme à 1) si activé.

Interprétation :

- **Diagonale forte :** Bon clustering - chaque type cellulaire correspond à un cluster distinct.
- **Hors diagonale :** Confusions - ex : si ligne « CD4 T » a 0.3 en colonne 2 et 0.7 en colonne 5, 30% des CD4 T sont mal classés.
- **Colonnes vides :** Clusters non documentés dans la vérité terrain.

12.6.4 Analyse Silhouette par Cluster

Visualisation

Diagramme en « couteau » montrant la distribution des scores de silhouette pour chaque cluster.

Génération : `viz.plot_cluster_silhouette(sample_silhouette_values, labels)` dans `utils/visualization.py`.

Calcul interne :

```
1 from sklearn.metrics import silhouette_samples
2 sample_scores = silhouette_samples(embeddings, labels)
```

Interprétation :

- Chaque bande horizontale représente un cluster.

- La largeur de la bande = taille du cluster.
- Cellules avec score positif (droite de 0) : bien classées.
- Cellules avec score négatif (gauche de 0) : probablement mal assignées.
- La ligne rouge verticale = score moyen global.
- **Cluster homogène** : Toutes les cellules ont des scores similaires et positifs.
- **Cluster flou** : Large dispersion ou valeurs négatives.

12.6.5 Boxplot Statistique (Comparaison Multi-Run)

Visualisation

Boxplots comparant la distribution des scores (NMI, ARI, ACC, Silhouette) sur N répétitions, avec lettres CLD.

Génération : `viz.plot_metrics_comparison(summary, results, show_stats=True)` dans `utils/visualization.py`.

Test statistique :

- **Kruskal-Wallis** (défaut, non-paramétrique) ou ANOVA.
- Post-hoc : Groupement par Compact Letter Display (CLD).
- Algorithmes partageant une même lettre ne sont pas significativement différents.

12.6.6 Inspecteur de Gènes par Groupe (Differential Expression)

L'outil « Group Gene Inspector » permet d'identifier les marqueurs biologiques de chaque cluster via une analyse d'expression différentielle rigoureuse.

Méthodologie :

- **Test Statistique** : Test de Wilcoxon (Rank-Sum) pour comparer l'expression dans le cluster cible vs le reste.
- **Correction** : Benjamini-Hochberg (FDR) sur les p-values.
- **Log2 Fold Change (Log2FC)** : Mesure l'ampleur de la surexpression.

Visualisation - Volcano Plot : Un graphique interactif (Log2FC en X, -Log10(P-val) en Y) permet d'identifier visuellement les gènes significatifs (en haut à droite pour les surexprimés).

12.6.7 Inspecteur de Batch (Data Upload)

Pour l'analyse de données multi-sources, l'onglet « Data Upload » propose un inspecteur de *batch* dynamique.

Fonctionnalités :

- Visualisation de la distribution des types cellulaires pour un batch spécifique.
- Détection des déséquilibres de classes (ex : absence d'un type cellulaire dans un batch).
- Métriques spécifiques au batch (nombre de cellules, nombre de classes).

12.6.8 Aides Contextuelles (Tooltips)

Pour faciliter l'adoption par des utilisateurs non-experts, l'interface intègre des aides contextuelles (icônes ?) détaillant :

- La signification biologique des métriques (ex : Top 10 Genes = part de lectures capturées).
- Les détails méthodologiques des tests statistiques.
- L'interprétation des graphiques complexes (Volcano plot, Heatmap).

12.7 Comparaison Train vs Test (Performance)

Visualisation

Graphique à barres comparant les scores (NMI, ARI, ACC) obtenus sur le jeu d'entraînement (bleu) vs le jeu de test (orange).

Interprétation :

- **Idéal** : Barres Test $\approx$ Barres Train (ou Test $>$ Train). Cela indique une bonne généralisation ; le modèle a appris des structures biologiques réelles et non du bruit.
- **Overfitting** : Train $\gg$ Test (ex : Train=0.9, Test=0.6). Le modèle a « par cœur » les données d'entraînement mais échoue sur de nouvelles données. Souvent dû à : trop d'époques, dimension latente trop grande, ou pas assez de régularisation.
- **Underfitting** : Train et Test sont tous deux faibles. Le modèle n'a pas assez de capacité ou les hyperparamètres ne sont pas adaptés.

12.8 Generalization Gap

Visualisation

Graphique représentant la différence $\Delta = \text{Score}_{\text{Train}} - \text{Score}_{\text{Test}}$.

Interprétation :

- **Barre Négative (vers le bas)** : Test $>$ Train. Le modèle obtient de meilleures performances sur le test que sur le train. Cela peut se produire lorsque le jeu de test contient des types cellulaires plus distincts ou moins bruités que le train, ou en présence de régularisation (dropout, masking).
- **Barre Positive (vers le haut)** : Train $>$ Test. Signe de perte de performance en généralisation.
- **Seuil d'alerte** : Un gap positif > 0.1 indique un sur-apprentissage sévère nécessitant une intervention (réduire la complexité, augmenter le dropout).

12.9 Métriques par Batch (Heatmap)

Visualisation

Tableau ou Heatmap montrant les scores NMI, ARI, ACC pour chaque batch individuel du jeu de test.

Interprétation :

- Permet d'identifier l'**hétérogénéité des données**.
- Si un batch a des scores systématiquement plus bas (ex : `human4` vs `human2`), cela suggère que ce batch est de moindre qualité technique (faible profondeur de séquençage, forts effets de batch) ou contient des types cellulaires rares non vus dans le train.
- Utile pour vérifier si le modèle est robuste aux effets de *batch* (« *batch invariant* »).

12.10 Analyse des Erreurs

Visualisation

Tableaux et Heatmaps montrant le taux d'erreur global, par batch, et les paires de confusion. **Visualisation Multi-Conditions** : Les heatmaps d'erreurs (`error_by_celltype_heatmap.png`) agrègent désormais automatiquement **tous les modes d'analyse** disponibles dans le dossier de résultats (ex : Standard, Split, Train on Batch X...). Cela permet une comparaison directe de la robustesse du modèle selon la stratégie d'entraînement (généralisation vs mémorisation).

Interprétation :

- **Taux d'erreur** : Pourcentage de cellules mal classées ($1 - \text{ACC}$).
- **Paires de confusion** (ex : True : 4, Pred : 0) : Indique quels types cellulaires sont biologiquement ou techniquement similaires.
- Si deux clusters sont constamment confondus, cela peut remettre en question la « vérité terrain » (annotation manuelle potentiellement erronée ou sub-clustering réel non annoté).

12.11 Projections UMAP

Visualisation

Scatter plot 2D des embeddings latents, coloré par labels réels ou prédits.

Visualisation Triple-Panneau (Comparaison) : Pour faciliter l'évaluation qualitative, SCRBenchmark génère automatiquement une figure à trois panneaux lors des analyses :

1. **Ground Truth** : Coloration par types cellulaires réels (labels de référence).
2. **Predicted Clusters (IDs)** : Coloration par les identifiants numériques bruts du modèle (0, 1, 2...). Utile pour voir la séparation « pure » de l'algorithme.
3. **Predicted (Aligned & Named)** : Coloration par les noms de types cellulaires après alignement optimal (Hungarian matching).

Interprétation :

- **Séparation** : Les clusters doivent former des îlots distincts. Des îlots fusionnés expliquent souvent les confusions.
- **Cohérence** : Les cellules d'un même type doivent être regroupées.
- **Effet de Batch** : Si l'on colore par batch, les points des différents batches doivent être mélangés (bon mélange) au sein de chaque cluster de type cellulaire. S'ils forment des sous-clusters séparés par batch, l'intégration a échoué.

13 Cohérence Interface Graphique et Ligne de Commande

Le logiciel garantit une cohérence complète entre les choix faits dans l'interface Streamlit et leur exécution, que ce soit via l'interface graphique ou la CLI.

13.1 Export de Configuration

L'export de configuration (JSON/YAML) dans l'onglet « Algorithm Config » inclut désormais :

- **input_type** : Type de données d'entrée choisi (`processed`, `raw_filtered`; `raw_original` est un mode legacy et est automatiquement ramené à `raw_filtered` dans l'UI/AnalysisRunner)
- Tous les hyperparamètres avec leurs valeurs actuelles

- Les métadonnées de l’algorithme (catégorie, GPU requis)

Ce champ `input_type` détermine si les algorithmes basés sur ZINB (scDeepCluster, scVI, scNAME, scMUG) reçoivent des comptes bruts ou des données prétraitées.

13.2 Mapping d’Architecture Réseau

Le *mapping* entre les paramètres d’architecture « unifiée » (Encoder/Decoder/Latent) et les paramètres spécifiques de chaque algorithme est maintenant synchronisé entre l’UI et la CLI.

Tableau de Correspondance Architecture

Algorithme	Encoder	Latent	Decoder
scDeepCluster	encoder_layers	z_dim	decoder_layers
Simple AE	hidden_layers	z_dim	decoder_layers
scCDCG	encoder_layers	embedding_dim	decoder_layers
scNAME/scMUG	hidden_dims (concat : encoder + latent)		
scHNTL	F1 (1er élément)	F2	(interne)
scMAE	(fixé)	hidden_size	(fixé)
scVI-DCT	n_hidden + n_layers	n_latent	(symétrique)

Exemple CLI :

```

1 ./scrbenchmark run --data data.h5ad --algorithms scdeepcluster,
   scvi_dct \
2   --encoder-layers 512,256 --z-dim 32
3 # Résultat :
4 # - scDeepCluster: encoder_layers="512,256", z_dim=32
5 # - scVI-DCT: n_hidden=512, n_layers=2, n_latent=32

```

13.3 Réinitialisation des Valeurs par Défaut

Lorsqu’un utilisateur efface un champ d’architecture dans l’UI, les valeurs par défaut de l’algorithme sont automatiquement restaurées :

- **scDeepCluster** : encoder=256,64, z_dim=32, decoder=64,256
- **Simple AE** : hidden=256,64, z_dim=32
- **scCDCG** : encoder=256, embedding=16
- **scNAME** : hidden_dims=256,64,32
- **scMUG** : hidden_dims=512,128,32
- **scHNTL** : F1=64, F2=16
- **scMAE** : hidden_size=128
- **scVI-DCT** : n_hidden=128, n_layers=2, n_latent=30

Cela évite les incohérences où l’UI affiche un champ vide mais l’ancienne valeur reste en mémoire de session.

13.3.1 Tableaux de Bord d’Analyse Bioinformatique

La plateforme propose des tableaux de bord spécialisés pour l’interprétation biologique :

- **DotPlot de Marqueurs Globaux** : Visualise la densité d’expression et les niveaux moyens des principaux gènes marqueurs à travers tous les clusters simultanément. Cela permet une validation biologique rapide de l’identité des clusters.

- **Analyse de Silhouette par Cluster** : Décompose le score de silhouette global en composantes individuelles par cluster, aidant à identifier les types cellulaires « flous » ou mal définis.
- **Heatmap de Performance par Batch** : En mode benchmark, les métriques sont ventilées par batch/dataset individuel, permettant aux utilisateurs d'évaluer la robustesse des algorithmes face aux variations techniques.
- **Coloration UMAP par Batch** : Permet une inspection visuelle de la qualité du mélange et de la correction des batches (mélange vs séparation des batches dans l'espace latent).

14 Reconstruction de Datasets et Gestion des Données Brutes

Une contribution majeure de ce projet est la mise en place d'un pipeline de reconstruction de datasets à partir des sources originales pour l'obtention des **raw counts** (comptages bruts). Cette étape est indispensable pour les modèles basés sur des distributions de probabilité discrètes (scDeepCluster, scVI, scMUG, scNAME) qui perdent en performance ou deviennent invalides sur des données normalisées (floats).

14.1 Problématique : Les Datasets de Référence

Les benchmarks standards ([?], OpenProblems) fournissent souvent des objets AnnData où la matrice **X** et les layers contiennent des données déjà normalisées par log-transformation ou correction de batch. Par exemple, dans le dataset `pancreas_openproblems.h5ad`, le layer `'counts'` contient en réalité des valeurs normalisées (ex : 1.0019583) et non des entiers.

14.2 Pipeline de Reconstruction (Cas du Pancréas)

Le script `scripts/setup/reconstruct_pancreas_raw_counts.py` permet de reconstruire intégralement le dataset Pancréas scIB en téléchargeant les 6 études originales (Baron, Muraro, Segerstolpe, Lawlor, Grün, Xin) et en effectuant une harmonisation rigoureuse.

14.2.1 Harmonisation des Identifiants de Gènes

Chaque étude utilise des conventions différentes pour nommer les gènes (Symbols, Ensembl, Entrez, ou concaténations avec le chromosome). Le pipeline utilise l'API **mygene** pour effectuer un mapping propre vers les symboles HGNC officiels :

```
1 import mygene
2 mg = mygene.MyGeneInfo()
3 results = mg.querymany(gene_ids, scopes='entrezgene', fields='symbol',
    , species='human')
```

14.2.2 Stratégies de Combinaison Génique

L'un des défis majeurs de l'intégration est la disparité du nombre de gènes mesurés entre technologies (ex : 20k pour Baron vs 40k pour Xin). Le logiciel propose deux stratégies via l'argument `-gene-strategy` :

- **Intersection** (Défaut) : On ne conserve que les gènes présents dans **tous** les datasets (~14,813 gènes). C'est la méthode la plus sûre car elle évite l'introduction de valeurs manquantes artificielles.
- **Union** : On conserve l'intégralité des gènes détectés (~51,501 gènes). Les valeurs manquantes dans les datasets n'ayant pas mesuré un gène sont remplies par des zéros. Cette approche maximise l'information mais augmente la taille du fichier et la sparsité.

14.3 Comparaison : Reconstruction vs Référence OpenProblems

La validation automatique du pipeline montre une fidélité quasi-totale avec la référence scIB, tout en apportant les données brutes manquantes.

TABLE 5 – Comparaison des spécifications du dataset Pancréas

Caractéristique	OpenProblems (Réf.)	Reconstruction (Inter.)
Nombre de Cellules	16,382	16,400
Nombre de Gènes	18,771	14,813
Types de données	Float (Normalisé)	Integer (Raw Counts)
Layer 'counts'	Normalisé	Vrais entiers

Note sur les Batches : Le batch `smarter` (Xin et al.) est inclus dans la reconstruction mais ne fournit que des données RPKM. Le logiciel émet un avertissement et recommande son exclusion pour les modèles ZINB via le flag `-exclude-batches smarter`.

15 Comparaison avec les Implémentations Originales

Les algorithmes intégrés dans `SCRBenchmark` (dossier `src/scrbenchmark/algorithms`) sont fidèles aux principes mathématiques et aux architectures des publications originales (dossier `external/original_code`). Cependant, des adaptations techniques majeures ont été nécessaires pour garantir un framework unifié, robuste et compatible multi-plateforme.

15.1 Standardisation Framework : Portabilité PyTorch

La majorité des algorithmes originaux étaient écrits en PyTorch, mais avec des styles disparates.

- **Cas Spécifique scNAME :** Le code original [?] est implémenté en **TensorFlow**. Nous avons entièrement ré-implémenté l'architecture (Mask Estimation, Contrastive Loss, ZINB) en **PyTorch** natif. Cette réécriture permet une intégration fluide (mêmes dépendances, même gestion mémoire) et l'exécution sur GPU/MPS sans conflits de frameworks.
- **Standardisation des I/O :** Les scripts originaux chargent souvent des fichiers spécifiques (CSV, h5ad avec conventions propriétaires). Dans `SCRBenchmark`, tous les algorithmes héritent de `BaseAlgorithm` et reçoivent un objet `AnnData` standardisé via le `DataHandler`, garantissant que tous les modèles voient exactement les mêmes données (mêmes filtres QC, mêmes HVGs).

15.2 Compatibilité Matérielle (Apple Silicon / MPS)

De nombreux codes de recherche originaux (ex : `scDeepCluster`, `scCDCG`) contiennent des configurations matérielles rigides incompatibles avec les environnements modernes comme les Mac M1/M2/M3 :

- **Types de données (Float64 vs Float32) :** `scDeepCluster` original force `torch.float64`, ce qui provoque des crashes immédiats sur le backend MPS (Metal Performance Shaders) d'Apple. Notre implémentation utilise `float32` par défaut pour assurer la compatibilité universelle (CPU, CUDA, MPS).
- **Hardcoded CUDA :** Les scripts originaux ciblent souvent `cuda:0` explicitement. Notre framework détecte dynamiquement le matériel disponible et alloue le device approprié sans intervention de l'utilisateur.

15.3 Rigueur Scientifique : Suppression des « Oracles »

Certains scripts originaux incluent des logiques d'évaluation biaisées pour maximiser artificiellement les scores dans leurs papiers :

- **Recherche « Oracle » de Résolution (scMAE, scNAME)** : Les codes originaux exécutent parfois le *clustering* de Leiden en boucle (boucle `while`) en ajustant la résolution jusqu'à trouver *exactement* le nombre de *clusters* correspondant à la vérité terrain (« *Ground Truth* »).
- **Correction Benchmark** : Cette pratique est interdite dans notre benchmark réaliste. Nous exposons ces hyperparamètres (résolution, nombre de clusters) à l'utilisateur ou utilisons des valeurs fixes. L'algorithme ne doit pas "voir" les étiquettes réelles pour optimiser ses hyperparamètres en interne, sauf si une procédure d'optimisation explicite sur le set de *validation* (et non test) est activée via le module `HyperparamSearch`.

15.4 Reproductibilité

Contrairement aux scripts "one-shot" de recherche qui omettent parfois de fixer toutes les sources d'aléatoire, SCRBenchmark fournit une méthode centralisée `BaseAlgorithm._set_seed()` et fixe explicitement les graines (`random`, `numpy`, `torch`, `cuda`) dans les algorithmes pour reproduire fidèlement les implémentations originales. Certaines méthodes appellent directement `_set_seed()`, d'autres répliquent la logique de seed à l'intérieur de `fit()`.

16 Étude de Cas : Le Pipeline scVI-DCTCorr + Enhanced AE + Leiden

Les benchmarks sur le dataset Baron Human Pancreas montrent que la combinaison **Enhanced Autoencoder + Leiden** appliquée sur des données corrigées par **scVI-DCTCorr** obtient les scores les plus élevés (NMI, ARI, ACC) parmi les configurations testées. Cette section décrit le fonctionnement de ce pipeline hybride tel qu'il est implémenté dans SCRBenchmark.

16.1 Architecture du Pipeline : scVI-DCTCorr → Enhanced AE → Leiden

Cette stratégie découple l'étape d'intégration des données (correction de batch) de l'étape de clustering. Par rapport au pipeline précédent (PCA + Leiden), la réduction PCA est remplacée par un auto-encodeur profond avec pondération par fréquence inverse.

16.2 Analyse des mécanismes

Cette architecture repose sur quatre composantes :

1. **Modélisation Probabiliste (scVI)** : scVI modélise le bruit de séquençage (dropout, overdispersion) via une distribution Binomiale Négative et génère une matrice d'expression débruitée.
2. **Contraintes d'alignement (DCTCorr)** : La Triplet Loss rapproche les cellules de même type issues de batches différents, tandis que la perte de corrélation (CorrMSE) préserve les relations inter-gènes propres à chaque batch.
3. **Représentation non-linéaire (Enhanced AE)** : L'auto-encodeur profond avec pondération par fréquence inverse apprend une représentation non-linéaire de l'espace d'expression. La pondération inverse attribue un poids plus élevé aux types cellulaires rares dans la fonction de perte.
4. **Détection de communautés (Leiden)** : Leiden détecte des communautés basées sur la densité locale du graphe KNN, ce qui ne suppose pas de forme sphérique des clusters (contrairement à K-Means).

Flux de Données Exact dans SCRBenchmark

1. Entrée : Comptes bruts (Raw Counts) non-normalisés.

Note : scVI modélise une distribution Negative Binomial et exige impérativement des entiers (fournis via `layers['original_X']`).

↓

2. Intégration (scVI-DCTCorr) : Entraînement d'un VAE (Variational Autoencoder) avec **Triplet Loss** (DCT) et **perte de corrélation** (CorrMSE) pour forcer l'alignement des types cellulaires identiques à travers les différents batches tout en préservant la structure corrélationnelle biologique.

↓

3. Reconstruction Denoised : Génération de la matrice d'expression corrigée $\hat{X}_{corrected}$ via le décodeur du modèle (μ_{post}).

Note : Cette étape supprime le dropout et le bruit technique, et retire l'effet batch de la matrice d'expression.

↓

4. Prétraitement standard : Normalisation, Log1p, sélection des HVGs et scaling sur $\hat{X}_{corrected}$.

↓

5. Injection : Remplacement de la matrice principale `adata.X` par cette version "propre".

↓

6. Encodage (Enhanced Autoencoder) : Compression des profils d'expression vers un espace latent de 64 dimensions via un auto-encodeur profond (512→256→128→64) avec entraînement en deux phases : warm-up (MSE standard) puis pondération par fréquence inverse des clusters pour améliorer la représentation des populations rares.

↓

7. Clustering (Leiden) : Construction du graphe KNN sur l'espace latent de l'auto-encodeur et détection de communautés par maximisation de la modularité.

16.3 Évolution par rapport au pipeline précédent

Par rapport au pipeline précédent (scVI-DCT + PCA + Leiden), deux modifications ont été apportées :

- **scVI-DCTCorr vs scVI-DCT** : L'ajout de la perte de corrélation ($\mathcal{L}_{CorrMSE}$) impose la conservation des corrélations inter-gènes propres à chaque batch (calculées via Per-Batch PCA, cf. section précédente) lors de la correction de batch.
- **Enhanced AE vs PCA** : Le remplacement de la PCA linéaire par un auto-encodeur profond avec pondération par fréquence inverse fournit une représentation non-linéaire qui attribue un poids plus élevé aux types cellulaires rares.

Cette configuration est accessible dans le logiciel en activant **Batch Correction** avec la méthode `scvi` dans l'onglet Data Handler, puis en sélectionnant l'algorithme **Enhanced AE + Leiden**. En CLI, l'activation se fait via `-batch-correction -batch-correction-method scvi`. Les poids DCT/CorrMSE sont paramétrés via la configuration (ou l'interface GUI).

16.4 Scripts de Benchmark Batch-Correction

Pour évaluer systématiquement les stratégies de correction de batch, deux scripts dédiés orchestrent des matrices d'expériences sur le dataset OpenProblems Pancreas :

- `run_openproblems_batch_correction_all_ssh.sh` : Version **supervisée** (`-batch-correction-method scvi`). Détecte automatiquement la colonne batch, la colonne de labels (pour la DCT) et

la cible d'équilibrage. Exécute 6 conditions expérimentales (*standard no-split*, *standard balanced*, *split no-balance*, *split balanced no-reinject*, *split balanced reinject*, *split correction*).

- `run_openproblems_batch_correction_unsupervised_all_ssh.sh` : Version **non-supervisée** (`-batch-correction-method scvi_unsupervised`). Même matrice de 6 conditions, mais sans détection de colonne de labels. Expose les hyperparamètres spécifiques : `pseudo-n-clusters` ($K=14$), `pseudo-kmeans-iters` ($I=5$), `triplet-warmup-epochs` ($W=30$).

Chaque script sélectionne automatiquement le GPU le plus libre, active un environnement virtuel dédié, et journalise l'ensemble de l'exécution. La comparaison des résultats supervisés et non-supervisés permet d'évaluer le coût (en performance de clustering) de l'absence de labels de types cellulaires lors de la correction de batch.

16.4.1 Conditions effectivement présentes dans `results_openproblems_pancreas_04_02`

Le dossier `results/results_openproblems_pancreas_04_02` contient 20 conditions (horodatées entre le 04/02/2026 et le 09/02/2026), issues du script de base `run_openproblems_benchmark.sh` et de relances ciblées sur les variantes avec correction scVI externe.

Paramètres communs observés dans les `config_used.json` :

- Dataset : `data/pancreas_raw_counts.h5ad`.
- Prétraitement : `n_top_genes=2000`, `min_genes_per_cell=1`, `min_cells_per_gene=1`.
- Répétitions : `n_repeats=5`.
- En mode split : `train=0.70`, `val=0.10`, `test=0.20`, stratification `batch+labels`.
- Cible d'équilibrage : 638 cellules (plus petit batch détecté automatiquement).

Les conditions se répartissent en quatre familles :

- **Sans split (standard)** : `standard_no_split_no_balanced`, `standard_no_split_balanced`, `standard_no_split_no_balanced_scvi_no_TL`, `standard_no_split_balanced_scvi_no_TL`.
- **Split multi-batch (train/test stratifié)** : `split_no_balance`, `split_no_balance_correction_scvi_no_TL`, `split_balanced_no_reinject`, `split_balanced_no_reinject_scvi_no_TL`, `split_balanced_reinject`, `split_balanced_reinject_scvi_no_TL`, `split_balanced_correction_scvi_no_TL`.
- **Split mono-batch (train sur un batch unique, test hors batch)** : `split_batch_celseq_reinject`, `split_batch_celseq2_reinject`, `split_batch_fluidigmcl_reinject`, `split_batch_inDrop1_reinject`, `split_batch_inDrop2_reinject`, `split_batch_inDrop3_reinject`, `split_batch_inDrop4_reinject`, `split_batch_smartseq2_reinject`.
- **Cas présent mais non exploitable** : `split_batch_smarter_reinject` (dossier créé, mais sans `config_used.json` archivé).

Du point de vue méthodologique, ces 20 dossiers couvrent trois axes expérimentaux orthogonaux :

- **Protocole de split** : standard (pas de split) vs benchmark train/val/test.
- **Équilibrage de batch** : aucun, élimination (`balance_mode=eliminate`) ou réinjection dans le train (`balance_mode=reinject_train`).
- **Correction externe** : sans correction (`do_batch_correction=false`) vs correction scVI externe (`do_batch_correction=true`, dossiers suffixés `_scvi_no_TL`).

Enfin, les jeux d'algorithmes diffèrent selon les familles :

- Conditions standards/split sans correction externe : panel large (incluant `scdeeplcluster`, `sc_mae`, `sccdcg`, `scname`, `enhanced_autoencoder`, et selon les cas `scvi_dct`).
- Conditions avec correction scVI externe (`_scvi_no_TL`) : panel restreint à `pca_kmeans`, `pca_leiden`, `simple_autoencoder`, `enhanced_autoencoder`.
- Conditions mono-batch : panel intermédiaire (identique au panel large, sans `scvi_dct`).

16.5 Alternative Non-Supervisée

La variante non-supervisée (`scvi_unsupervised`) s'applique aux datasets sans annotation cellulaire préalable. Les cas d'usage incluent :

- Les données exploratoires où les types cellulaires ne sont pas encore caractérisés.
- Les analyses à grande échelle où l'annotation manuelle n'est pas disponible.
- L'évaluation de la robustesse du pipeline : comparer les performances avec et sans labels permet de mesurer la dépendance du modèle aux annotations.

La visualisation **Composition batch Train/Test** du Results Explorer affiche la répartition des cellules par batch et type cellulaire dans chaque split, permettant d'identifier les biais potentiels d'apprentissage (ex : un type cellulaire présent uniquement dans un batch du train).

17 Validation Automatique et « Guardrails » Scientifiques

Pour garantir la rigueur scientifique quelle que soit la configuration choisie par l'utilisateur (CLI ou GUI), nous avons implémenté des « *Scientific Guardrails* » stricts :

- **Validation des Données d'Entrée** : Le système vérifie automatiquement si l'algorithme demande des données brutes (ex : `scVI`, `scDeepCluster`) ou prétraitées. Si des données normalisées (flottants) sont fournies à un modèle nécessitant des comptes bruts (ZINB), une erreur explicite est levée pour empêcher la génération de résultats invalides.
- **Compatibilité Benchmark** : Les algorithmes transductifs basés sur graphes (`scMUG`, `scHNTL`) ne supportent pas la prédiction *out-of-sample*. Le cœur du pipeline les bloque en vrai Train/Test, tandis que le GUI/CLI les exécute en *fallback* transductif (entraînement + évaluation sur l'ensemble complet). `scCDCG` est **inductif** et reste autorisé en *benchmark*.
- **Correction de Batch scVI** : Le pipeline de prétraitement *Benchmark* a été durci pour s'assurer que la correction de *batch* par `scVI` utilise systématiquement les comptes bruts (`original_X`), même si l'utilisateur a sélectionné des données prétraitées pour le reste de l'analyse.

18 Conclusion

Ce rapport a présenté SCRBenchmark, une plateforme pour l'évaluation comparative d'algorithmes de *clustering* sur données scRNA-seq. Les éléments suivants ont été décrits :

1. **Cadre d'évaluation** : SCRBenchmark regroupe, via un registre extensible, des algorithmes classiques et *deep learning* dans un même cadre d'évaluation, avec des métriques standardisées (NMI, ARI, ACC, Silhouette) et des métriques de robustesse (UCA, F1_Macro, BalancedACC, RareACC, KNN Purity, NNO), ainsi qu'un protocole de *benchmarking* avec séparation train/test stratifiée.
2. **Correction de batch effects** : Le logiciel intègre deux méthodes de correction de *batch* robustes : `scVI-DCT` (supervisé) et `sysVI` (cycle-consistency + VampPrior), adaptées aux contextes de forte hétérogénéité technique.
3. **Pipeline retenu** : Parmi les configurations testées sur le dataset Baron Human Pancreas, le pipeline `scVI-DCTCorr` + **Enhanced Autoencoder** + **Leiden** obtient les scores les plus élevés. Cette architecture combine la modélisation probabiliste (`scVI`) pour le débruitage, la perte de corrélation (CorrMSE) pour la conservation de la structure inter-gènes, un auto-encodeur profond avec pondération par fréquence inverse, et l'algorithme Leiden pour la détection de communautés.

4. **Validations automatiques** : Des mécanismes de validation (« *guardrails* ») vérifient la cohérence des données d’entrée, la compatibilité des algorithmes avec le mode *benchmark*, et l’utilisation des comptes bruts pour les modèles probabilistes.
5. **Interface double** : L’outil est accessible via une interface graphique Streamlit (exploration interactive) et une interface en ligne de commande (analyses en lot).

Perspectives. Plusieurs axes sont envisagés : l’intégration de nouveaux algorithmes (notamment les approches fondées sur les *transformers*), l’extension du *benchmark* à des données multi-omiques (CITE-seq, ATAC-seq), et l’ajout d’un module d’optimisation bayésienne des hyperparamètres. La validation sur des cohortes cliniques à grande échelle permettrait de tester le pipeline retenu dans un contexte translationnel.

Références

- [1] Hubert, L., & Arabie, P. (1985). *Comparing partitions*. Journal of Classification, 2(1), 193–218.
- [2] Strehl, A., & Ghosh, J. (2002). *Cluster ensembles—a knowledge reuse framework for combining multiple partitions*. Journal of Machine Learning Research, 3(Dec), 583–617.
- [3] Kuhn, H. W. (1955). *The Hungarian method for the assignment problem*. Naval Research Logistics Quarterly, 2(1–2), 83–97.
- [4] Xie, J., Girshick, R., & Farhadi, A. (2016). *Unsupervised deep embedding for clustering analysis*. International Conference on Machine Learning (ICML), 478–487.
- [5] Traag, V. A., Waltman, L., & van Eck, N. J. (2019). *From Louvain to Leiden : guaranteeing well-connected communities*. Scientific Reports, 9(1), 5233.
- [6] Lopez, R., Regier, J., Cole, M. B., Jordan, M. I., & Yosef, N. (2018). *Deep generative modeling for single-cell transcriptomics*. Nature Methods, 15(12), 1053–1058.
- [7] Tian, T., Wan, J., Song, Q., & Wei, Z. (2019). *Clustering single-cell RNA-seq data with a model-based deep learning approach*. Nature Machine Intelligence, 1(4), 191–198.
- [8] Fang, Z., Fan, R., & Zeng, J. (2024). *scMAE : a masked autoencoder for single-cell RNA-seq clustering*. Bioinformatics, 40(1), btad716.
- [9] Wan, H., Chen, L., & Deng, M. (2022). *scNAME : neighborhood contrastive clustering with ancillary mask estimation for scRNA-seq data*. Bioinformatics, 38(6), 1575–1583.
- [10] Meng, X., Zhang, X., & Li, M. (2025). *schNTL : single-cell clustering via high-order neighbors and triplet loss*. Briefings in Bioinformatics, 26(1), bbae640.
- [11] Liang, D. & Du, P. (2025). *scMUG : a framework leveraging multiple gene functional modules for single-cell clustering*. Genome Biology, 26, 56.
- [12] Xu, J., Xu, J., Meng, Y., & Lu, C. (2024). *scCDCG : efficient deep structural clustering for single-cell RNA-seq via deep cut-informed graph embedding*. arXiv preprint arXiv :2404.06167.
- [13] Luecken, M. D., Büttner, M., Chaichoompu, K., et al. (2022). *Benchmarking atlas-level data integration in single-cell genomics*. Nature Methods, 19(1), 41–50.